Titre du rapport

GUILHEM

2026-07-03

Contents

1 Introduction . . . 3
2 Contexte et objectifs . . . 4
3 Méthodologie expérimentale . . . 4

4 Multiplication matricielle **4**
4.1 Algorithmes étudiés . . . 4
4.2 Tailles et configurations . . . 5
4.3 Immensité de l'espace de configurations . . . 5
4.4 Analyse détaillée des optimisations . . . 5
A noter : . . . 5
4.4.1 Optimisations compilateur . . . 5
4.4.2 Vectorisation . . . 6
4.4.3 MPI . . . 6
4.4.4 OpenMP . . . 6
4.4.5 Blocking . . . 6
4.4.6 Transposition . . . 7
4.5 Résultats globaux . . . 7
4.5.1 Classement des optimisations les plus efficaces . . . 7
4.6 Analyse mémoire / cache / TLB / IPC . . . 7
4.7 Scalabilité . . . 8
4.8 Énergie et puissance . . . 8
4.9 Comparaison SUMMA / Fox / Cannon . . . 8
4.10 Conclusion locale . . . 8
4.11 Perspectives . . . 9

5. Modèle SEIR **9**
5.1 Présentation du modèle . . . 9

5.2 Analyse du modèle SEIR **10**

5.2.1 Analyse de l'algorithme sequentiel **10**

5.2.1.1 Analyse des optimisations 10

5.2.1.2 Incidence du pourcentage d'individus infecté (cf annexe : SEIR_A): 10

5.2.1.3 Comparaison des differentes vectorisations possibles 11

5.2.1.4 Différences liées à la taille des structures utilisées. 12

5.2.2 Analyse des versions MPI: **13**

5.2.2.1 Analyse de la taille des structures : 13

5.2.2.2 Importance des optimisations en MPI : 14

5.2.2.3 Scalabilité en MPI : 15

5.2.2.3 Vectorisation en MPI 16

5.2.3 Analyse des algorithmes OpenMP : **16**

5.2.3.1 Analyse des optimisations O0 - Ofast **16**

5.2.3.2 Taille des structures 17

5.2.3.3 Différences entre les deux versions OpenMp 18

5.2.4 Analyse des l'algorithmes hybrid (MPI+OpenMP) **19**

5.2.4.1 Comapraison des deux algorithme hybrid 19

5.2.4.2 Scalabilité de la version hybrid 20

5.3 ANALLYSE GLOBALE DES RESULTATS **21**

5.4 Conclusion finale du modèle SEIR **22**

6 Jeu De La Vie **23**

6.1 Présentation **23**

6.2 Analyse **24**

6.2.1 Analyse de l'algorithme sequentiel **24**

6.2.1.1 Analyse des optimisations 24

6.2.1.2 Analyse de l'impact du blocking: 25

6.2.1.2 Coeur de calcul C1 vs C2 sur sequentielle blocké 26

6.2.1.3 Cœurs sur la version séquentielle non bloquée : 27

6.2.1.4 Vectorisation en sequentiel 28

6.2.2 Paradigme MPI : **29**
6.2.2.1 Scalabilité . . . 29
6.2.2.2 Incidence du blocking par rapport à la version non bloquée : . . . 30
6.2.2.3 Coeurs de calculs en MPI : . . . 31
6.2.2.4 Synthèse générale sur les coeurs : . . . 31
6.2.2.5 Vectorisation sur MPI . . . 32

6.2.3 Paradigme OpenMP : **33**
6.2.3.1 Blocking . . . 33
6.2.3.2 Scalibilité en OpenMP . . . 34

6.2.3.3 Blocking dans OpenMP **35**

6.2.3.4 Vectorisation en OpenMP **36**

6.3 Comparaison des differents paradigmes **37**

6.3.1 Dans leur version par default **37**

6.3.2 Dans leur meilleurs configuration : **38**

7 Conclusion **39**
7.1 Résumé synthétique . . . 39
7.2 Tendances et interprétation technique . . . 39
7.3 Recommandations pratiques . . . 39
7.4 Note personnelle . . . 40
7.5 Limites et perspectives . . . 40
7.6 Bilan final . . . 40

Annexes **41**

Annexe **42**
Annexe SEIR_A . . . 42

1 Introduction

Ce rapport rassemble une série d'études expérimentales visant à évaluer l'impact de plusieurs optimisations logicielles et architecturales — **MPI**, **OpenMP**, **transposition**, **blocking**, **vectorisation** et **optimisations de compilation** — sur des codes scientifiques parallèles. L'analyse porte principalement sur la **multiplication matricielle dense**, complétée par deux études complémentaires (modèle SEIR et Jeu de la vie) afin de dégager des tendances générales. Les métriques étudiées sont le **temps d'exécution**, l'**énergie**, la **puissance** et la **scalabilité**, pour différentes tailles de problème et configurations. L'objectif est d'identifier les techniques réellement efficaces sur une architecture mono-nœud et de comprendre leurs interactions pratiques.

2 Contexte et objectifs

Les campagnes de tests ont été réalisées sur une machine équipée d'un **AMD Ryzen AI 7 350 (8 cœurs / 16 threads)**. Les paramètres explorés incluent la taille des matrices, le nombre de processus/threads, les tailles de blocs, les schémas de communication MPI et les options de compilation. L'objectif est double : (i) déterminer quelles optimisations apportent les gains les plus significatifs dans ce contexte matériel, (ii) caractériser les limites et interactions entre optimisations afin de proposer des recommandations pratiques pour l'optimisation de codes numériques sur une machine mono-nœud.

---

3 Méthodologie expérimentale

3.1 Plateforme matérielle Les tests ont été effectués sur un processeur **AMD Ryzen AI 7 350** (8 cœurs / 16 threads). La taille de page mémoire utilisée est **4 KiB (4096 bytes)**, un paramètre influençant la localité des accès et le comportement des optimisations mémoire (transposition, blocking). L'énergie du package CPU est mesurée via l'interface exposée par le noyau Linux (`/sys/class/powercap/intel-rapl:0/energy_uj`) ; cette lecture fournit une estimation de l'énergie du package CPU (cœurs, caches, logique interne). **Remarque** : la disponibilité et le chemin d'accès à cette interface peuvent varier selon l'architecture et la version du noyau ; les valeurs présentées sont donc spécifiques à la machine testée.

3.2 Protocole de mesure et limites Après un ou deux runs d'échauffement, chaque configuration est exécutée **5 fois** avec un temps de pause de 1s entre chaque execution, avant d'en faire une moyenne pour les analyses qui en découlent. Pour les versions MPI, une synchronisation via `MPI_Barrier` encadre la section mesurée et les temps sont relevés avec `MPI_Wtime` (on retient le temps du processus le plus lent) ; pour les versions séquentielles et OpenMP, `clock_gettime` est utilisé. L'énergie est lue immédiatement **avant** et **après** la période mesurée via le compteur RAPL ; l'énergie consommée est la différence des lectures et la puissance moyenne est calculée par énergie/temps. Les mesures distinguent systématiquement **temps/énergie global** (initialisation → finalisation) et **temps/énergie algorithmique** (seulement la phase de calcul). Les conditions expérimentales sont contrôlées : machine dédiée, autres applications fermées, ventilation active et absence d'interaction pendant les benchs.

Limites : les mesures proviennent d'une seule machine et la couverture RAPL porte principalement sur le package CPU (la consommation DRAM peut ne pas être entièrement incluse). Les chemins d'accès RAPL peuvent différer sur d'autres plateformes ; pour faciliter la reproductibilité, les sources, Makefiles, scripts d'exécution et données brutes seront fournis dans le dépôt associé au rapport.

Après avoir défini le protocole et ses limites, passons à la description des algorithmes et des variantes testées.

---

4 Multiplication matricielle

4.1 Algorithmes étudiés

Les algorithmes testés couvrent un large spectre d'approches. Les versions séquentielles incluent la version **naïve**, la version **bloquée** et la version **transposée**. Les versions parallèles incluent **OpenMP**, OpenMP avec blocking, **MPI naïf**, MPI avec blocking, MPI avec vectorisation, et MPI avec hybridation OpenMP.

Les algorithmes de grille incluent **SUMMA**, **Fox** et **Cannon**, qui répartissent les matrices sur une grille de processus et utilisent des schémas de communication spécifiques pour effectuer le calcul. Les variantes *mlt* incluent `mult_send_rcv_replace`, `mult_bcast_blk` et `mult_mat_prl_blk`, qui reposent sur des schémas de communication plus simples mais souvent très efficaces.

4.2 Tailles et configurations

Les tests ont été réalisés sur des matrices de tailles **512, 1024, 2048, 4096 et 8192**. Les nombres de processus utilisés incluent **1, 2, 4, 8, 9, 12 et 16** selon les contraintes des implémentations. Les optimisations de compilation testées incluent **O0, O1, O2, O3** et **Ofast**.

Les tailles de bloc testées incluent **1, 8, 16, 32, 64, 128** et **1024**. Les tests ont été réalisés avec et sans transposition, avec vectorisation automatique (inhérente à O3), manuelle (utilisation des registre avx), avec vectorisation forcé (-ftree-vectorize), et vectorisation désactivée (fno-tree-vectorize), ainsi qu'avec ou sans hybridation MPI+OpenMP.

4.3 Immensité de l'espace de configurations

Même en se limitant à un ensemble raisonnable de paramètres, l'espace total des configurations reste extrêmement vaste. En combinant les algorithmes (5), les nombres de processus (3), les tailles de matrices (6), les tailles de blocs (6), les niveaux d'optimisation du compilateur (5), les modes OpenMP (2), les configurations de vectorisation (3) et trois types de données (3), on obtient déjà :

$$5 \times 3 \times 6 \times 6 \times 5 \times 2 \times 3 \times 3 = 97\,200 \text{ configurations}$$

Avec **5 répétitions** par configuration pour obtenir des mesures fiables : **486 000** exécutions.

En supposant un temps d'exécution optimiste de **5 secondes** par test, une campagne exhaustive représenterait déjà **23 jours** de calcul continu.

Il est donc nécessaire de sélectionner un sous-ensemble représentatif de configurations pour mener une analyse réaliste et pertinente.

4.4 Analyse détaillée des optimisations

A noter :

Les gains indiqués ci-dessous correspondent à des valeurs typiques observées sur les échantillon de tests. Les valeurs reportées sont des moyennes représentatives sur l'échantillons analysé (voir annexe/dépôt pour les graphiques détaillés).

4.4.1 Optimisations compilateur

Les optimisations de compilation apportent des gains modestes, généralement compris entre **10 % et 20 %**. La version **O0** présente des performances très faibles, tandis que **O1** offre une amélioration limitée. Les niveaux **O2** et **O3** constituent les options les plus stables et les plus efficaces, avec des gains réguliers sur l'ensemble des tailles de matrices.

La version **Ofast**, bien qu'en théorie plus agressive, présente des résultats très variables selon les configurations : elle peut parfois surpasser **O3**, mais aussi entraîner des régressions. Dans l'ensemble, **O3** représente le meilleur compromis entre performance et stabilité, et constitue le choix le plus pertinent pour les tests de performance.

4.4.2 Vectorisation

La vectorisation manuelle n'apporte aucun gain significatif, ce qui montre que le compilateur applique déjà une vectorisation automatique efficace. Les versions **MPI+AVX** présentent des performances identiques aux versions MPI classiques, confirmant que l'ajout explicite d'instructions vectorielles ne modifie pas le comportement du programme.

La vectorisation automatique optimise correctement les boucles internes, en exploitant les unités SIMD du processeur. Elle constitue une optimisation utile, mais ne nécessite aucune intervention manuelle, et son impact reste limité par rapport aux optimisations mémoire ou parallèles.

4.4.3 MPI

Les algorithmes MPI présentent une scalabilité correcte jusqu'à **9 processus**, mais souffrent d'une surcharge de communication lorsque le nombre de processus augmente. Sur une matrice de taille **4096**, les temps typiques sont **13 s** pour 4 processus, **5 s** pour 9 processus, et **5,5 s** pour 16 processus.

Les algorithmes de grille (SUMMA, Fox, Cannon) suivent une scalabilité logique, mais leur efficacité est fortement dépendante du coût des communications. Les variantes *mlt* sont très rapides, mais ne scalent pas, ce qui suggère une absence de parallélisation effective dans ces variantes.

Enfin, la version « MPI propre » présente des performances identiques à la version naïve, tout en offrant un code plus lisible.

4.4.4 OpenMP

OpenMP constitue l'une des optimisations les plus efficaces. Il permet d'exploiter pleinement les capacités multi-threads du processeur, en répartissant le travail sur plusieurs cœurs. Sur une matrice de taille **8192**, OpenMP réduit le temps d'exécution à **11,6 s**, contre **35 s** pour la version MPI (configuration testée).

OpenMP bénéficie particulièrement du **blocking**, qui améliore la localité mémoire et réduit les défauts de cache. La combinaison **OpenMP + blocking** constitue la meilleure optimisation globale, avec des gains pouvant atteindre un **facteur $\times$8**.

OpenMP profite également de la vectorisation automatique, mais la vectorisation manuelle n'apporte aucun gain supplémentaire.

4.4.5 Blocking

Le **blocking** constitue une optimisation efficace, mais son impact dépend fortement de la présence ou non de transposition. Sur les matrices non transposées, il permet de réduire le temps d'exécution d'un facteur pouvant atteindre **$\times$3**, et la consommation énergétique d'environ **$\times$2,8**, à condition d'utiliser une taille de bloc adaptée.

Cette taille optimale dépend de plusieurs paramètres : dimension de la matrice, type de données (int, short), taille des pages mémoire, et caractéristiques de l'architecture. Sur les matrices transposées, le blocking est moins efficace, car la transposition optimise déjà les accès mémoire.

Dans les versions OpenMP, le blocking constitue une optimisation particulièrement efficace, avec des gains pouvant atteindre **$\times$8** dans certaines configuration. Dans les versions MPI, il apporte des gains significatifs, généralement compris entre **$\times$3** et **$\times$5**.

4.4.6 Transposition

La **transposition** constitue l'optimisation la plus efficace de toutes celles testées. Elle améliore drastiquement la localité mémoire, réduit les défauts de cache, et permet au compilateur d'optimiser plus efficacement les accès. Sur une matrice de taille **8192**, l'algorithme SUMMA passe de **30–40 s** à seulement **3,6 s** lorsque la transposition est utilisée.

La transposition réduit également la consommation énergétique, en diminuant les stalls mémoire et en augmentant l'efficacité du CPU. Elle entraîne toutefois une augmentation de la puissance moyenne, qui peut atteindre **45 W**, contre **15 W** pour la version non transposée.

Elle constitue donc une optimisation essentielle pour les grandes matrices, et un prérequis pour obtenir des performances élevées sur les architectures modernes.

4.5 Résultats globaux

4.5.1 Classement des optimisations les plus efficaces

Les résultats obtenus permettent d'établir une hiérarchie claire des optimisations : (gain exprimé par rapport à la version sans la méthode décrite)

- **Transposition** — optimisation la plus efficace, gains jusqu'à $\times$**8**.
- **OpenMP** — gains importants, entre $\times$**5** et $\times$**8**.
- **MPI** — gains modérés, entre $\times$**2** et $\times$**4**.
- **Blocking** — gains significatifs, entre $\times$**2** et $\times$**4** selon les cas.
- **Optimisations compilateur** — gains modestes, entre **10 %** et **20 %**.
- **Vectorisation manuelle** — aucun gain notable.

Cette synthèse confirme que les optimisations mémoire et les approches multi-threads sont les plus efficaces sur une architecture mono-nœud, tandis que les optimisations de compilation et la vectorisation manuelle jouent un rôle secondaire.

4.6 Analyse mémoire / cache / TLB / IPC

Les défauts de cache et TLB restent faibles, ce qui indique que le goulot d'étranglement n'est pas la mémoire mais le calcul et la communication MPI. Les défauts de cache sont généralement inférieurs à **1 %**, tandis que les défauts de TLB sont inférieurs à **0,5 %**.

Le taux de défauts de TLB d'instructions est légèrement plus élevé, généralement compris entre **3 %** et **4 %**, ce qui suggère que le code occupe une partie significative du cache d'instructions.

L'IPC baisse après optimisation, passant de **2** à **1,5**, ce qui suggère que le compilateur réordonne les instructions pour réduire les stalls. Cependant, le temps d'exécution diminue malgré la baisse d'IPC, ce qui indique que l'IPC n'est pas un indicateur absolu de performance.

4.7 Scalabilité

La scalabilité dépend fortement de la taille de la matrice. Sur les petites matrices, les versions MPI présentent une scalabilité limitée, en raison de la surcharge de communication par rapport à la quantité de calculs à effectuer. Sur les matrices de taille **1024**, les versions MPI présentent une efficacité faible, généralement inférieure à **52 %**.

Sur les matrices de taille **2048**, les versions MPI présentent une scalabilité correcte, avec une efficacité maximale pour **9 processus**. Sur les matrices de taille **4096** et **8192**, les versions MPI présentent une scalabilité limitée en raison de la surcharge de communication. Les performances avec **16 processus** s'améliorent légèrement.

Les versions OpenMP présentent une scalabilité excellente, en raison du partage mémoire uniforme et de l'absence de coût de communication.

4.8 Énergie et puissance

L'énergie dépend pour ce modèle presque uniquement du temps d'exécution. La corrélation entre le temps et l'énergie est de **0,97**, ce qui suggère que l'énergie est proportionnelle au temps. Les versions *mlt* présentent une consommation énergétique faible du fait de leur implémentation les rendant aussi très performantes.

Les versions SUMMA, Fox et Cannon présentent une consommation énergétique élevée. Les versions OpenMP présentent une consommation énergétique plus faible.

La puissance est stable, généralement comprise entre **6 W** et **13 W**. Les versions transposées présentent une puissance plus élevée, généralement comprise entre **15 W** et **45 W**, du fait de la forte vectorisation des calculs.

4.9 Comparaison SUMMA / Fox / Cannon

SUMMA constitue le meilleur compromis entre performance et simplicité. Il présente une scalabilité correcte, une consommation énergétique modérée, une complexité de communication raisonnable, et surtout une implémentation facilitée par sa topologie.

Fox présente des performances légèrement meilleures en temps réel, mais souffre d'une surcharge de communication qui se répercute sur les temps d'exécution. **Cannon** présente un bon équilibrage.

4.10 Conclusion locale

Les résultats montrent que la **transposition** constitue l'optimisation la plus efficace, suivie par **OpenMP** et le **blocking**. Les versions MPI présentent une scalabilité limitée, en raison de la surcharge de communication. Les variantes *mlt* sont très rapides, mais étonnamment ne scalent pas ; cela est probablement dû au fait que, pour simplifier les communications, une matrice B transposée a été utilisée, ce qui explique leur avantage dans certains cas.

La vectorisation manuelle n'apporte aucun gain significatif, ce qui suggère que la vectorisation automatique du compilateur est déjà optimale. Les optimisations de compilation apportent des gains modestes, généralement compris entre **10 %** et **20 %**.

L'énergie dépend presque uniquement du temps d'exécution, tandis que la puissance est stable et peu discriminante.

4.11 Perspectives

Plusieurs pistes peuvent être envisagées pour étendre ce travail. Il serait intéressant d'étendre les tests à des architectures **multi-nœuds**, afin d'évaluer l'impact de la communication inter-nœuds. Il serait également pertinent d'étudier l'impact énergétique des accès mémoire, qui n'est ici que peu considéré en l'absence d'un compteur DRAM sur la machine de test.

5. Modèle SEIR

5.1 Présentation du modèle

Le modèle SEIR répartit la population en quatre états (S, E, I, R) et simule l'évolution d'une épidémie selon la dynamique S→E→I→R. Dans notre implémentation, la population est représentée sur une grille où les individus se déplacent localement à chaque itération, ce qui facilite l'usage de différents paradigmes (séquentiel, OpenMP, MPI, hybride). Le modèle dépend de trois paramètres : la taille de la grille, le nombre d'individus et le pourcentage initial d'infectés.

Toutes les versions reposent sur la même logique : déplacement des individus, mise à jour synchronisée de l'état via deux grilles (propagation locale de l'infection). Les différences portent uniquement sur la manière d'exploiter le parallélisme :

Séquentiel : parcours naïf de la grille, sans parallélisme.

OpenMP : V1: parallèlise les deux phases de chaque itération (cause beaucoup de dépendance), V2 paralélise uniquement la partie de mise à jour des états.

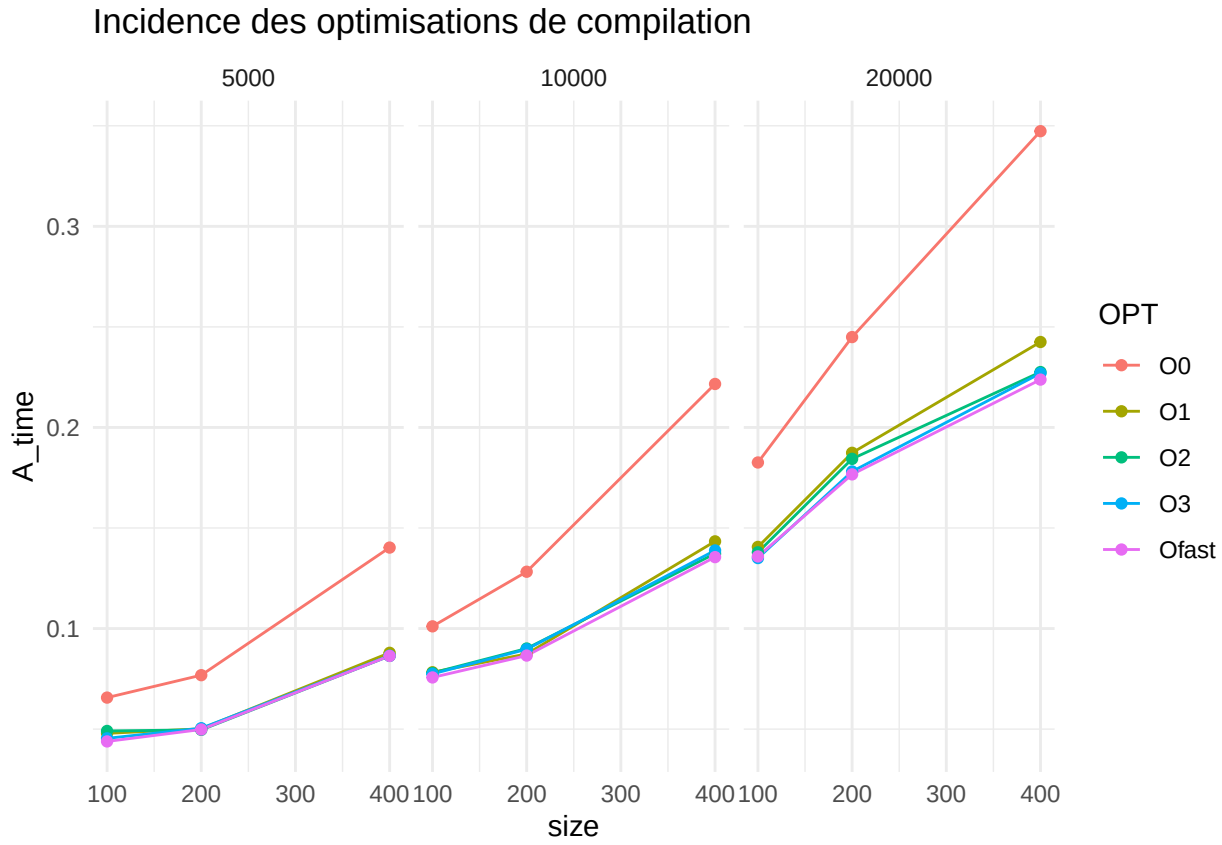
MPI : découpage de la grille en sous-domaines avec échanges de bordures.

Ce rapport analyse l'impact des optimisations du compilateur, de la vectorisation, de la taille des structures mémoire et compare les performances des quatre paradigmes. Maintenant que le cadre est posé nous pouvons à présent commencer l'analyse, regardons tout d'abord l'effet des optimisations O0 - Ofast du compilateur sur la version séquentielle:

5.2 Analyse du modèle SEIR

5.2.1 Analyse de l'algorithme sequentiel

5.2.1.1 Analyse des optimisations



Les différentes optimisations du compilateur présentent des performances globalement similaires, à l'exception notable du niveau O0, nettement en retrait et pouvant être jusqu'à deux fois plus lent que les meilleures configurations. Les niveaux O3 et Ofast offrent les meilleurs résultats et restent très proches en termes de rapidité, mais O3 se distingue par une meilleure stabilité. Pour la suite des benchmarks, nous retenons donc O3 comme optimisation de référence.

5.2.1.2 Incidence du pourcentage d'individus infecté (cf annexe : SEIR_A):

Le pourcentage initial d'infectés n'a aucun impact sur les performances : le coût de calcul dépend du nombre total d'individus, indépendamment de leur état.

5.2.1.3 Comparaison des differentes vectorisations possibles

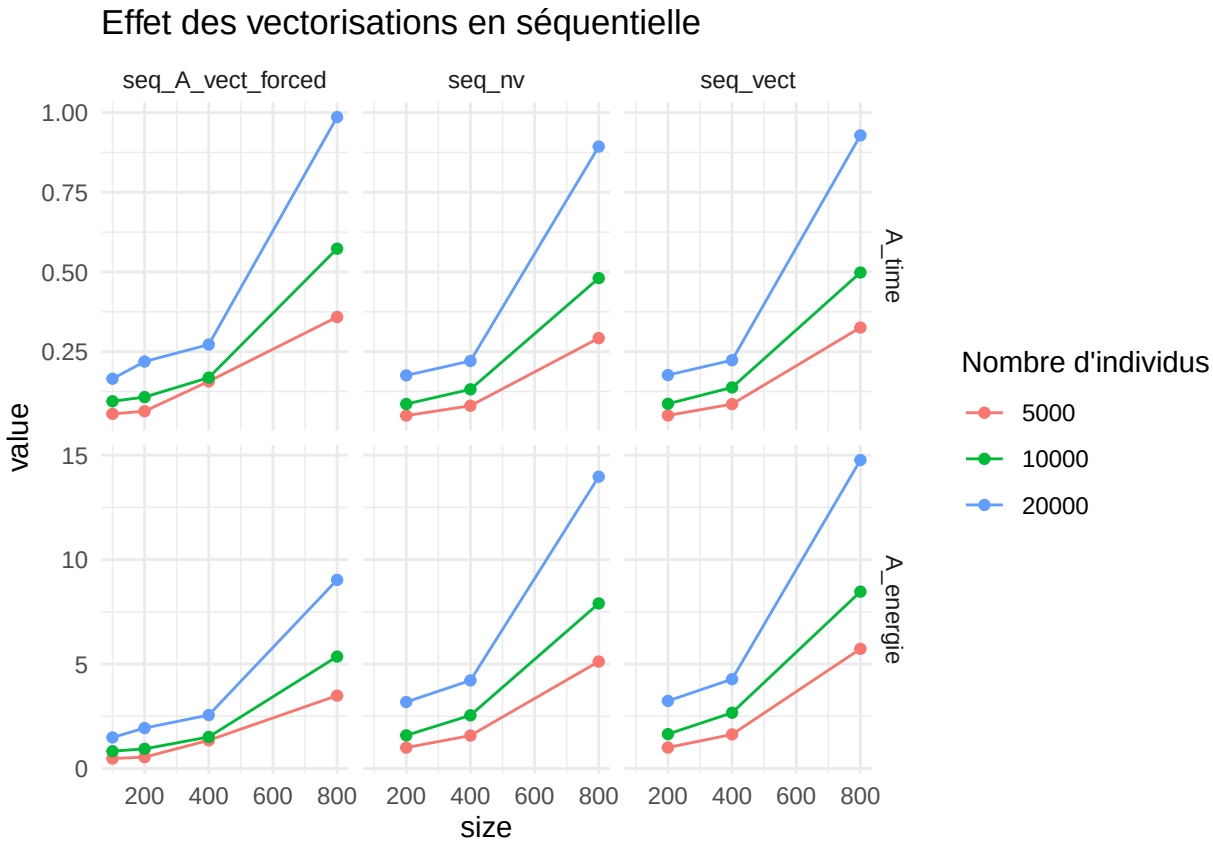


Table 1: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
seq_A_vect_forced	0.8113584	0.6175609	15
seq_nv	1.0369519	0.9582077	9
seq_vect	1.0000000	1.0000000	9

Toutes les versions sont compilées en O3. La version seq_vect correspond au comportement par défaut : le compilateur vectorise automatiquement lorsque c'est possible. La version seq_nv est compilée avec -fno-tree-vectorize, ce qui désactive toute vectorisation. Enfin, la version seq_A_vect_forced utilise -ftree-vectorize, qui pousse le compilateur à vectoriser de manière agressive.

Les graphiques montrent le temps d'exécution en haut et la consommation énergétique en bas. Deux résultats importants se dégagent.

1. Temps d'exécution : Les différences entre les versions restent modestes, de l'ordre de 10 %. La version non vectorisée est légèrement plus rapide, tandis que la vectorisation forcée est un peu plus lente. Pour ce modèle SEIR, cela indique que la vectorisation n'améliore pas la vitesse, et peut même introduire une petite pénalité.

2. Consommation énergétique : La tendance est inverse : la version à vectorisation forcée est nettement plus économe, avec un gain d'environ 38 % par rapport à la version par défaut. Autrement dit, même si elle est un peu plus lente, elle consomme beaucoup moins d'énergie.

3. Analyse des ratios : Le tableau des ratios confirme ces observations : la vectorisation forcée est environ 20 % plus lente, mais 40 % plus économe. Le gain énergétique compense largement la perte de performance. La version non vectorisée montre une tendance similaire, mais beaucoup plus faible, et les écarts restent dans la marge d'erreur.

5.2.1.4 Différences liées à la taille des structures utilisées.

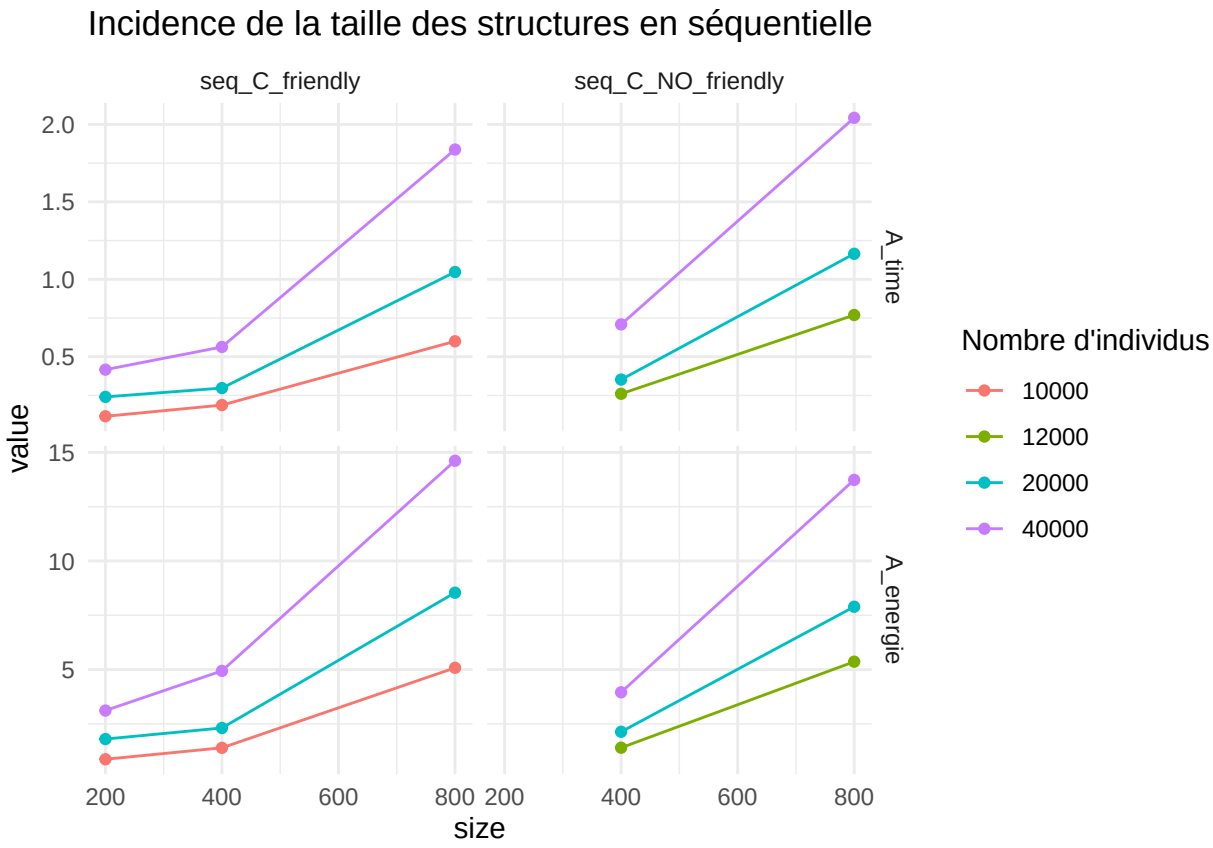


Table 2: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
seq_C_NO_friendly	0.8959053	0.9328808	16
seq_C_friendly	1.0000000	1.0000000	15

La réduction de la taille des structures améliore logiquement le temps d'exécution, mais s'accompagne d'une légère hausse de la consommation d'énergie, ce qui peut sembler contre-intuitif. Les ratios montrent que la version NO_friendly est environ 11 % plus lente, pour seulement ~7 % d'économie d'énergie, alors que les deux variantes ne diffèrent que d'environ 20 % en taille de structure.

Ces résultats indiquent que la consommation d'énergie est peu sensible à la taille des structures : réduire la structure n'abaisse presque pas l'énergie consommée, mais améliore le temps d'exécution de manière proportionnelle (environ la moitié du gain lié à la réduction de taille). Dans ces conditions, la variante plus volumineuse n'apporte aucun avantage réel : elle augmente la pression mémoire et pourrait même entraîner un coût énergétique global plus élevé dans des scénarios plus complexes, ce qui annulerait le faible gain observé.

5.2.2 Analyse des versions MPI:

5.2.2.1 Analyse de la taille des structures :

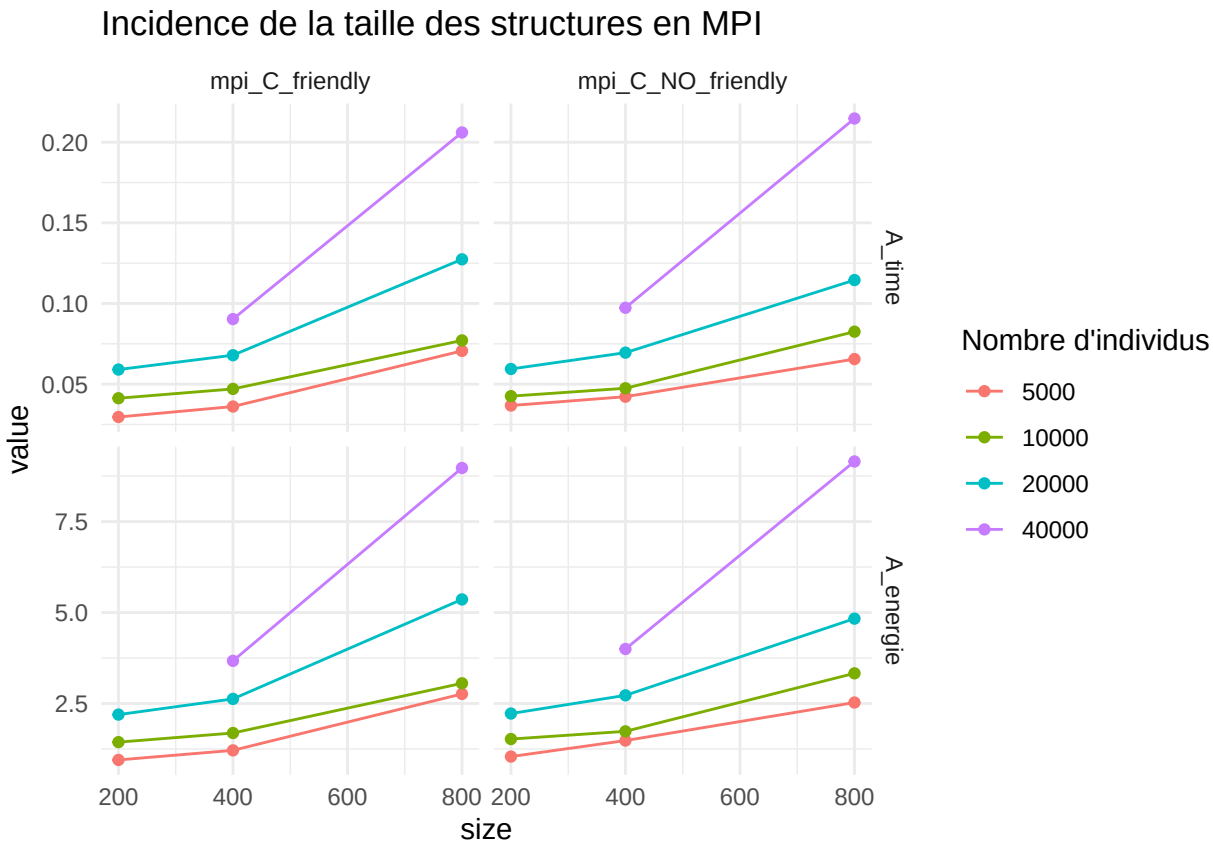


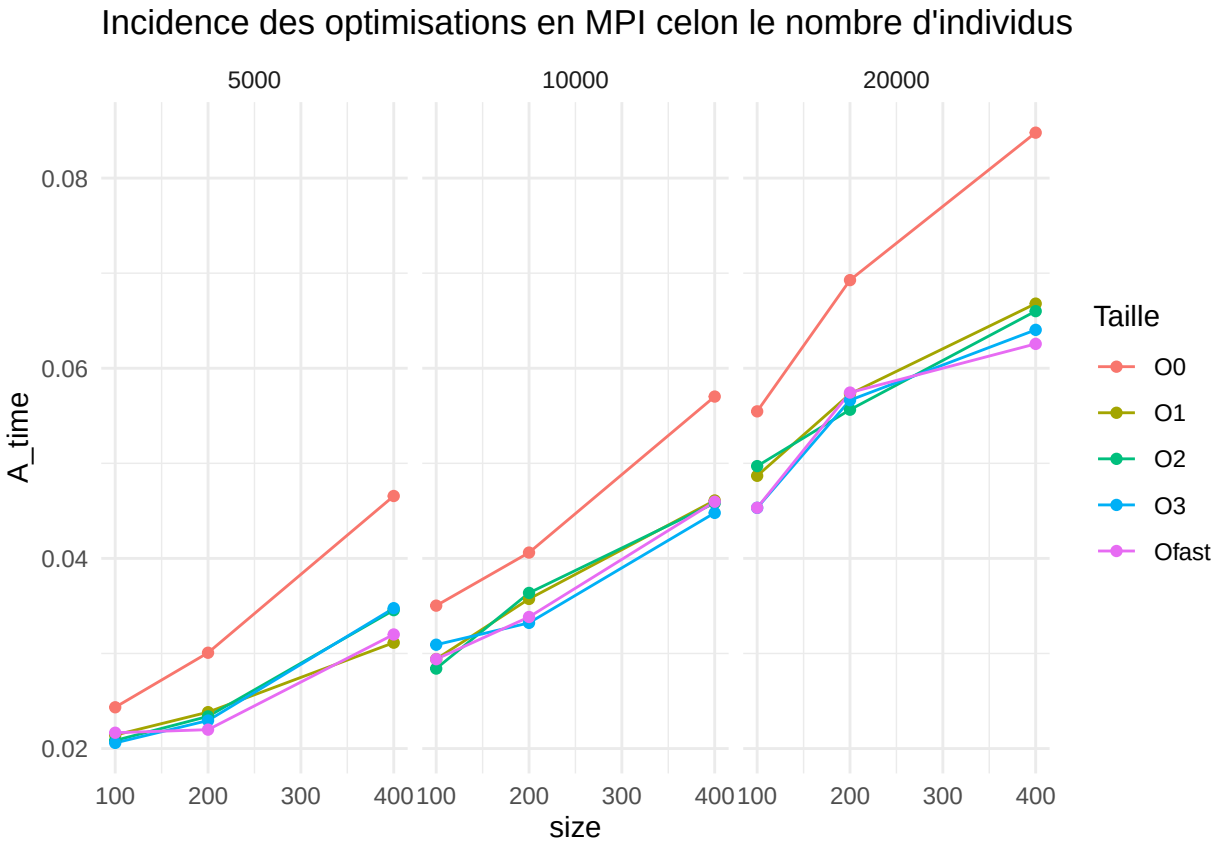
Table 3: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
mpi_C_NO_friendly	0.9974731	1.001972	81
mpi_C_friendly	1.0000000	1.000000	81

Dans la version MPI, la taille des structures ne semble avoir aucun impact significatif ni sur le temps d'exécution, ni sur la consommation d'énergie. Les légères différences observées, généralement à l'avantage de la version cache_friendly, restent clairement dans la marge d'erreur.

Cela s'explique par le fait que, dans MPI, le coût dominant provient du découpage de la grille, des communications entre processus et de la synchronisation des bordures. Ces opérations masquent largement les effets liés à la taille des structures internes. Ainsi, contrairement à la version séquentielle, la pression mémoire locale joue un rôle très limité dans les performances globales du modèle.

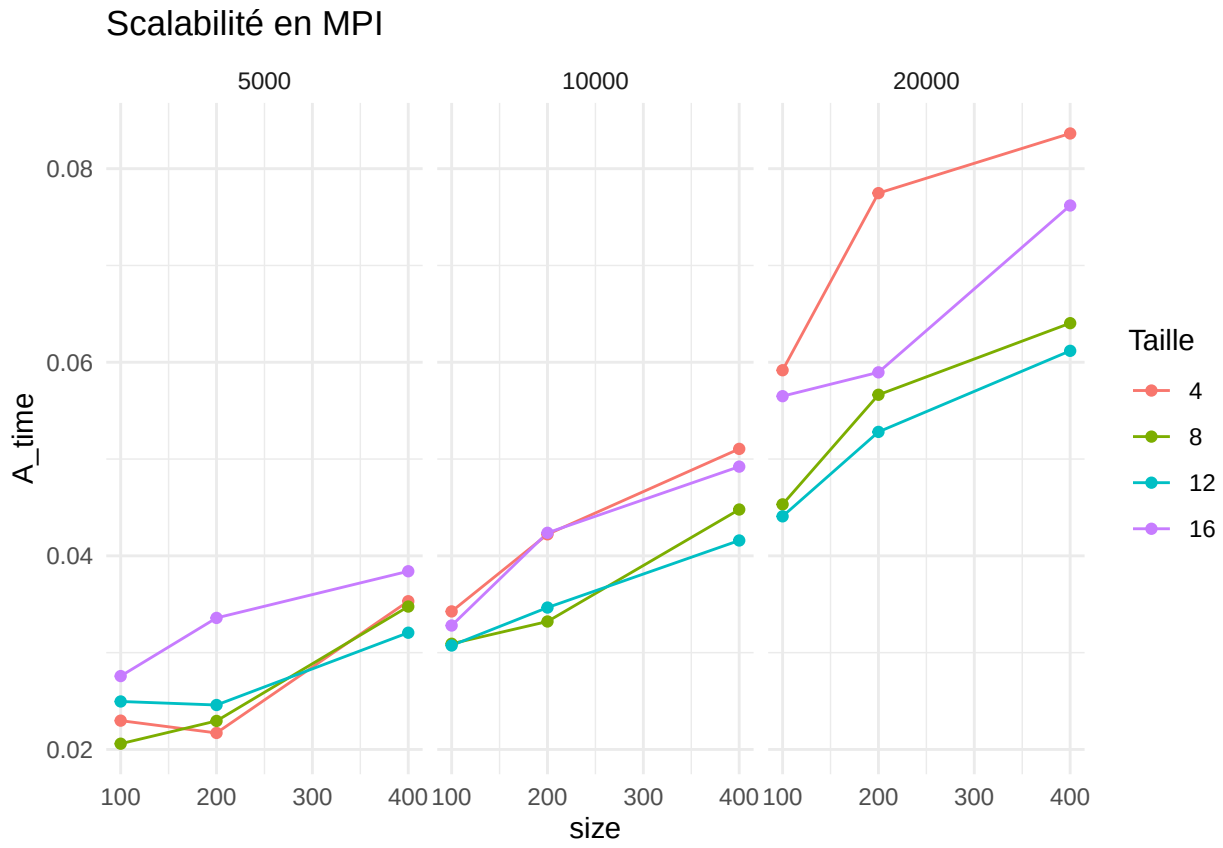
5.2.2.2 Importance des optimisations en MPI :



Optimisations du compilateur en MPI Les résultats sont un peu moins tranchés que pour la version séquentielle, mais la tendance générale reste la même. Le niveau O0 est clairement en retrait, tandis que O3 offre globalement les meilleures performances. Les autres niveaux d'optimisation se situent entre les deux, sans différence marquée.

Ainsi, même en MPI — où le coût des communications domine souvent le temps de calcul — O3 demeure l'option la plus efficace, confirmant son statut d'optimisation de référence pour la suite des benchmarks. Regardon à present la scalabilité du probleme avec l'utilisation de MPI

5.2.2.3 Scalabilité en MPI :



On observe qu'il existe un nombre optimal de processus à ne pas dépasser. La scalabilité est bonne jusqu'à 12 processus, mais passer à 16 dégrade nettement le temps d'exécution. La consommation d'énergie suit exactement la même tendance : le point optimal est également atteint à 12 processus, à la fois en performance et en efficacité énergétique.

Il est également notable que, pour certaines tailles de problème très petites, il peut être préférable d'utiliser 8, voire 4 processus, car au-delà, le coût de communication devient dominant et annule les bénéfices du parallélisme.

5.2.2.3 Vectorisation en MPI

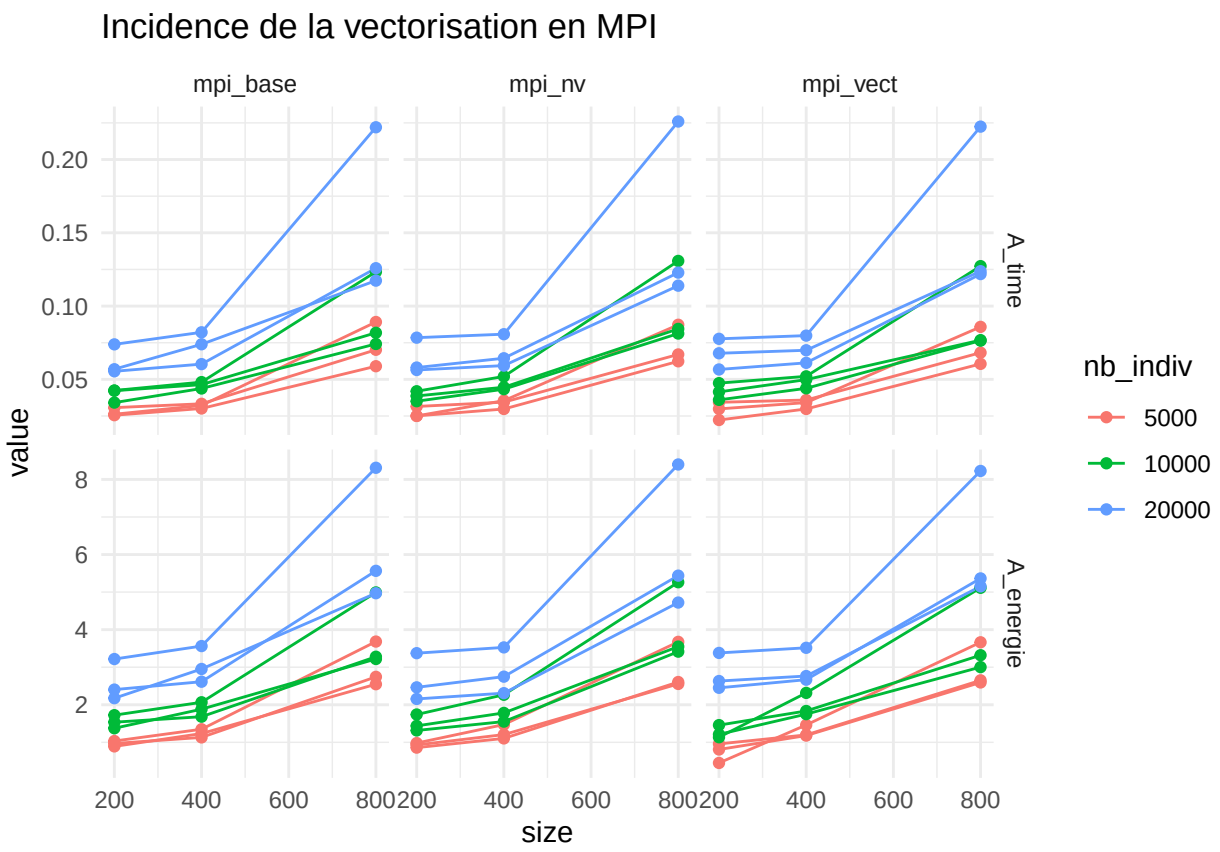


Table 4: Résumé des ratios MPI (vect vs no_vect vs basique)

exec	speedup	energy_ratio	n
mpi_base	1.0000000	1.0000000	27
mpi_nv	0.9986819	0.9917306	27
mpi_vect	0.9772205	0.9719590	27

Dans la version MPI, les écarts entre les différentes configurations de vectorisation sont beaucoup plus faibles que dans la version séquentielle. On retrouve globalement la même tendance — la vectorisation forcée apporte un léger avantage énergétique et une petite pénalité en temps — mais les différences sont ici très réduites.

Étant donné la rapidité d'exécution des programmes MPI et le poids dominant des communications et de la synchronisation dans le coût total, ces variations se situent à la limite de la marge d'erreur. Autrement dit, sous MPI, la vectorisation a un impact bien moindre que dans le séquentiel, et son effet devient presque négligeable.

5.2.3 Analyse des algorithmes OpenMP :

5.2.3.1 Analyse des optimisations O0 - Ofast

Concernant l'impact des optimisations du compilateur, on retrouve exactement les mêmes tendances que pour les versions MPI et séquentielles. Le niveau O0 reste nettement en retrait, tandis que O3 fournit

systématiquement les meilleures performances et la meilleure stabilité.

Les résultats étant parfaitement cohérents avec les analyses précédentes et ne montrant aucune information nouvelle, nous ne présentons pas ici les graphiques correspondants.

5.2.3.2 Taille des structures

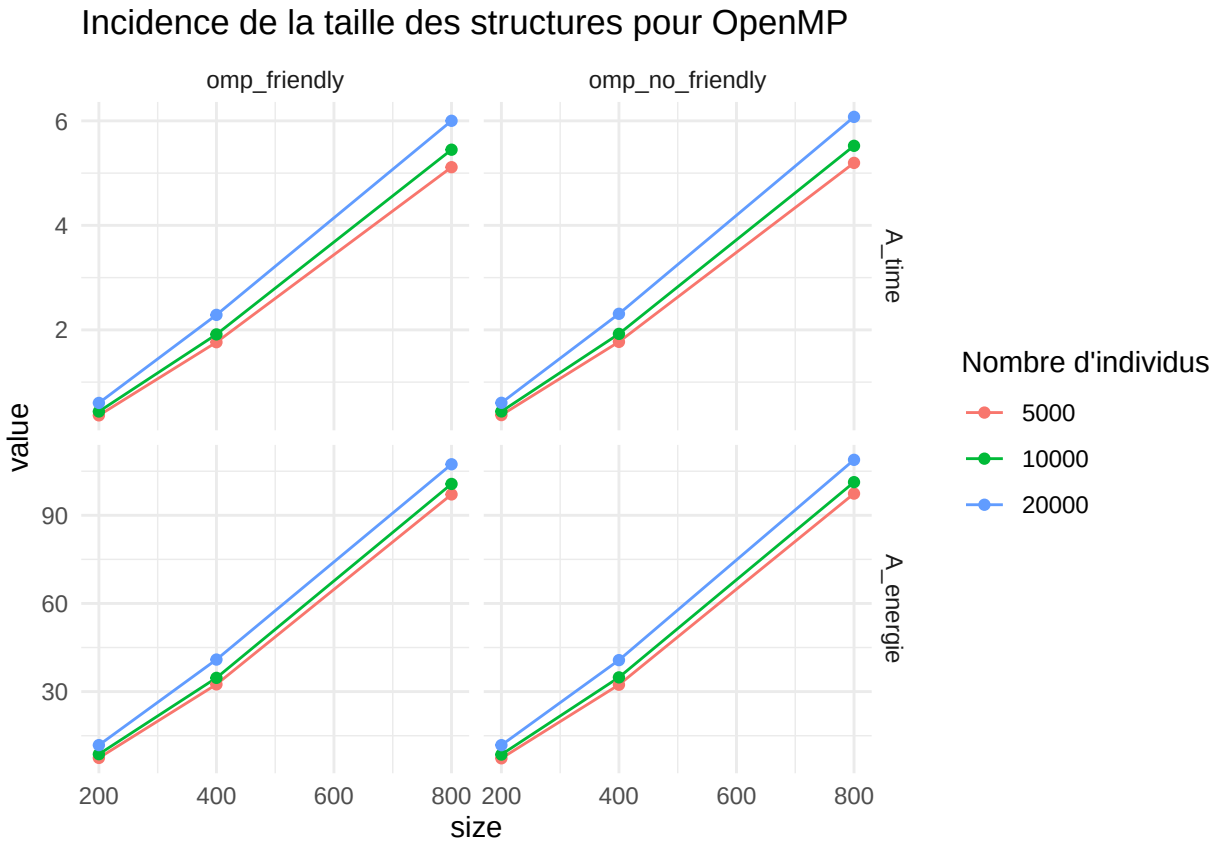


Table 5: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
omp_friendly	1.0000000	1.0000000	9
omp_no_friendly	0.9909516	0.9984437	9

Pour OpenMP, nous ne constatons aucune différence notable entre les différentes tailles de structures. Les temps d'exécution comme la consommation d'énergie restent pratiquement identiques, et les écarts observés se situent clairement dans la marge d'erreur.

Cela confirme que, dans OpenMP, la taille des structures a un impact négligeable : le coût dominant provient de la gestion du parallélisme (synchronisation, répartition des tâches), ce qui masque les effets liés à la pression mémoire.

5.2.3.3 Différences entre les deux versions OpenMp

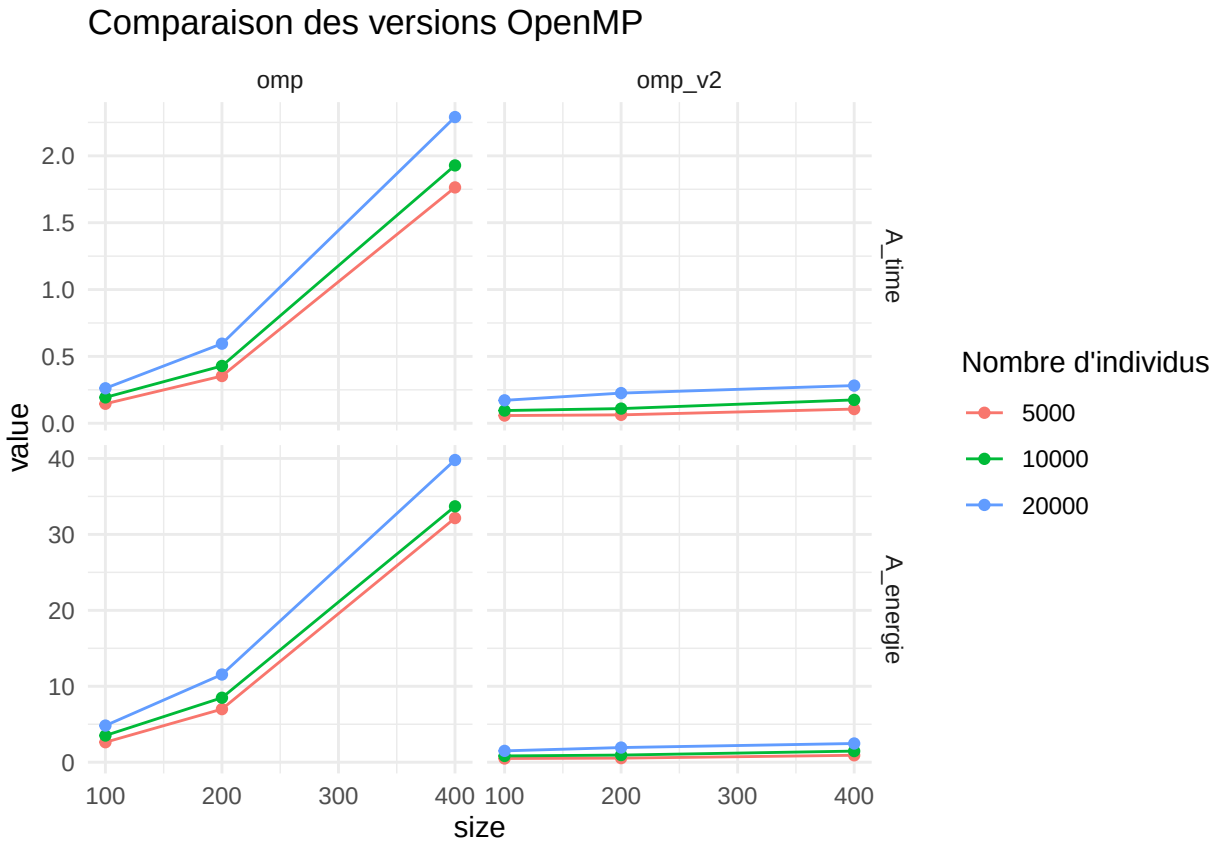


Table 6: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
omp	1.000000	1.0000000	9
omp_v2	6.009729	0.1356445	9

On voit très clairement que la version qui force la parallélisation (OMP V1) est vraiment très mauvaise par rapport à la version 2 (OMP V2), qui ne parallélise que ce qu'elle peut sans introduire de dépendances. Le coût de calcul est nettement plus faible dans OMP V2, et la consommation énergétique est réduite dans les mêmes proportions.

Ici, les ratios sont particulièrement marquants : un speed-up d'environ $\times 6$ entre V1 et V2, une consommation énergétique divisée par ~ 7.

La version OMP V1 est donc à oublier, tout simplement.

On voit qu'il est tentant de vouloir paralléliser un maximum de choses, mais dans ce cas précis, tout ce qui est mis en place dans V1 pour permettre à OpenMP de paralléliser (notamment l'enchaînement de malloc/free pour isoler les données entre threads) introduit une surcharge énorme. Cela ne permet absolument pas d'augmenter la vitesse d'exécution — au contraire, cela la dégrade fortement.

Cette chute de performance dans OMP V1 est probablement liée au fait que les allocations mémoire (malloc/free) ne sont pas parallélisables : elles passent toutes par un allocateur global protégé par un verrou. Les threads OpenMP doivent donc attendre les uns après les autres, ce qui annule totalement le bénéfice du

parallélisme et introduit une surcharge massive. Cela explique pourquoi OMP V1 est beaucoup plus lente et plus énergivore que OMP V2.

5.2.4 Analyse des l'algorithmes hybrid (MPI+OpenMP)

5.2.4.1 Comapraison des deux algorithme hybrid

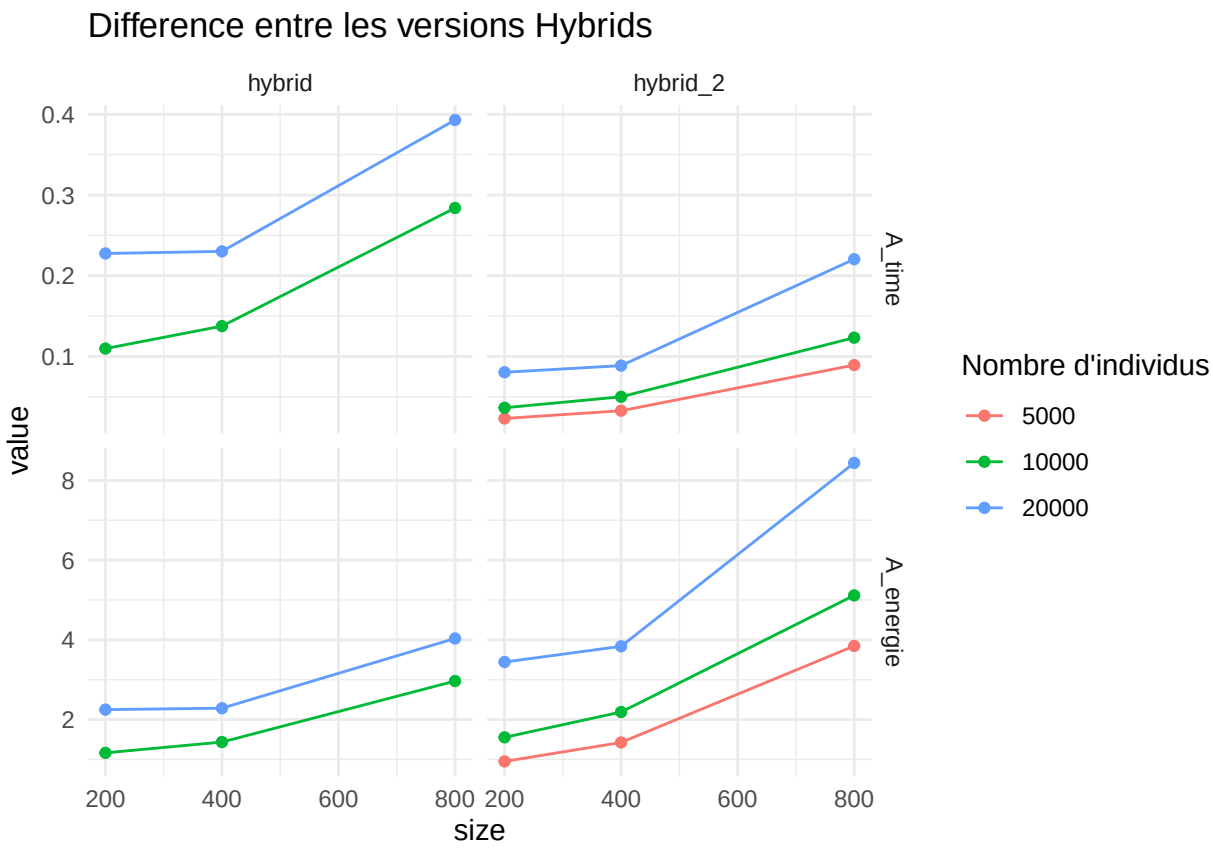


Table 7: Résumé des ratios par algorithme

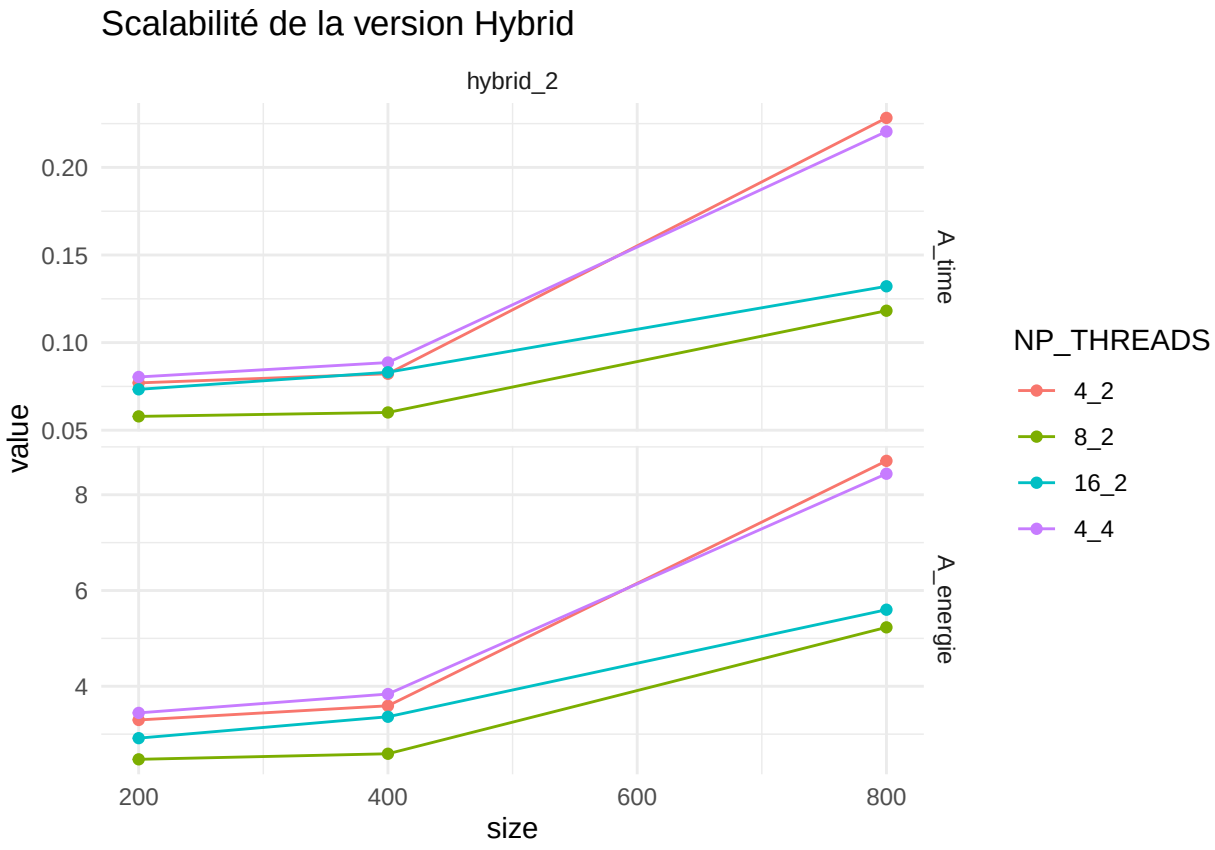
exec	speedup	energy_ratio	n
hybrid	1.000000	1.000000	6
hybrid_2	2.545958	1.647844	9

Les résultats mettent en évidence deux comportements intéressants. La version hybrid_2, basée sur omp_v2, est nettement plus rapide, avec un speedup d'environ 2,5. En revanche, elle consomme environ 64 % d'énergie de plus que la version hybride classique. Cette différence s'explique par le fait que la V2 passe beaucoup moins de temps en attente et parvient à paralléliser efficacement les sections utiles, ce qui active davantage les unités vectorielles (AVX). Or, même lorsque les registres SIMD ne sont pas entièrement remplis — ce qui peut arriver avec des tailles de données modestes — l'unité AVX consomme autant d'énergie tout en produisant moins de résultats par instruction. La V2 sollicite donc le CPU de manière plus intensive, ce qui augmente sa consommation totale malgré un temps d'exécution réduit.

En pratique, la V2 offre un meilleur compromis temps d'exécution / énergie, mais dans des contextes où la priorité est la sobriété énergétique, la V1 peut rester pertinente.

Il faut toutefois ajouter une nuance importante : les mesures énergétiques reposent sur RAPL, qui ne comptabilise essentiellement que l'énergie du CPU. Les coûts liés à la mémoire ne sont pas tous intégrés. Or, la version V1 effectue de très nombreux malloc/free, ce qui augmente la pression mémoire et devrait, en réalité, pénaliser davantage sa consommation totale. Ainsi, la différence énergétique observée en faveur de la V1 est probablement sous-estimée, car une partie de son coût réel n'est pas mesurée.

5.2.4.2 Scalabilité de la version hybrid



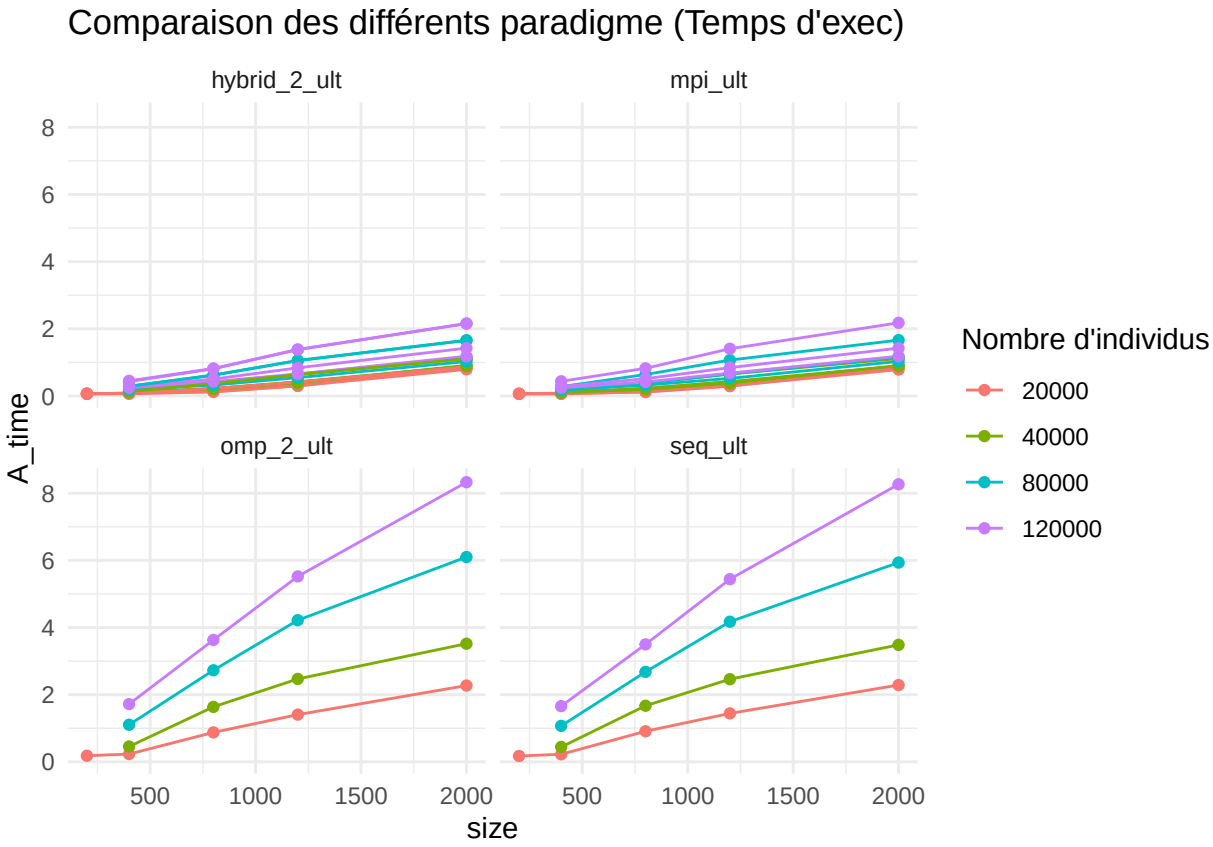
On observe que pour la version hybrid_v2, le temps d'exécution est très fortement corrélé à l'énergie consommée. La meilleure configuration est clairement 8 processus MPI / 2 threads OpenMP, qui produit simultanément les courbes les plus basses en temps et en énergie.

Avec np = 4, on voit que pour les deux plus petites tailles, il reste intéressant de conserver omp = 2, mais à partir de size = 800, il devient préférable d'augmenter le nombre de threads OpenMP : le temps diminue, et comme le temps est presque parfaitement corrélé à l'énergie dans cet échantillon, la consommation énergétique diminue également.

On observe également un point intéressant : lors d'un lancement involontaire avec np = 16 et omp = 2, soit 32 processus au total alors que la machine utilisée n'en posède que 16, donc deux fois plus que le nombre de cœurs physiques disponibles, les performances ne s'effondrent pas complètement. Cette configuration reste légèrement moins bonne que le couple optimal 8 MPI / 2 OMP, mais l'écart reste étonnamment faible. Cela montre que, dans ce modèle, l'oversubscription ne pénalise pas autant qu'on pourrait l'attendre, probablement parce que les sections parallèles sont suffisamment courtes pour que la surcharge de scheduling reste contenue.

À noter que la version hybrid_v1 montre exactement la même tendance, avec les mêmes con

5.3 ANALLYSE GLOBALE DES RESULTATS



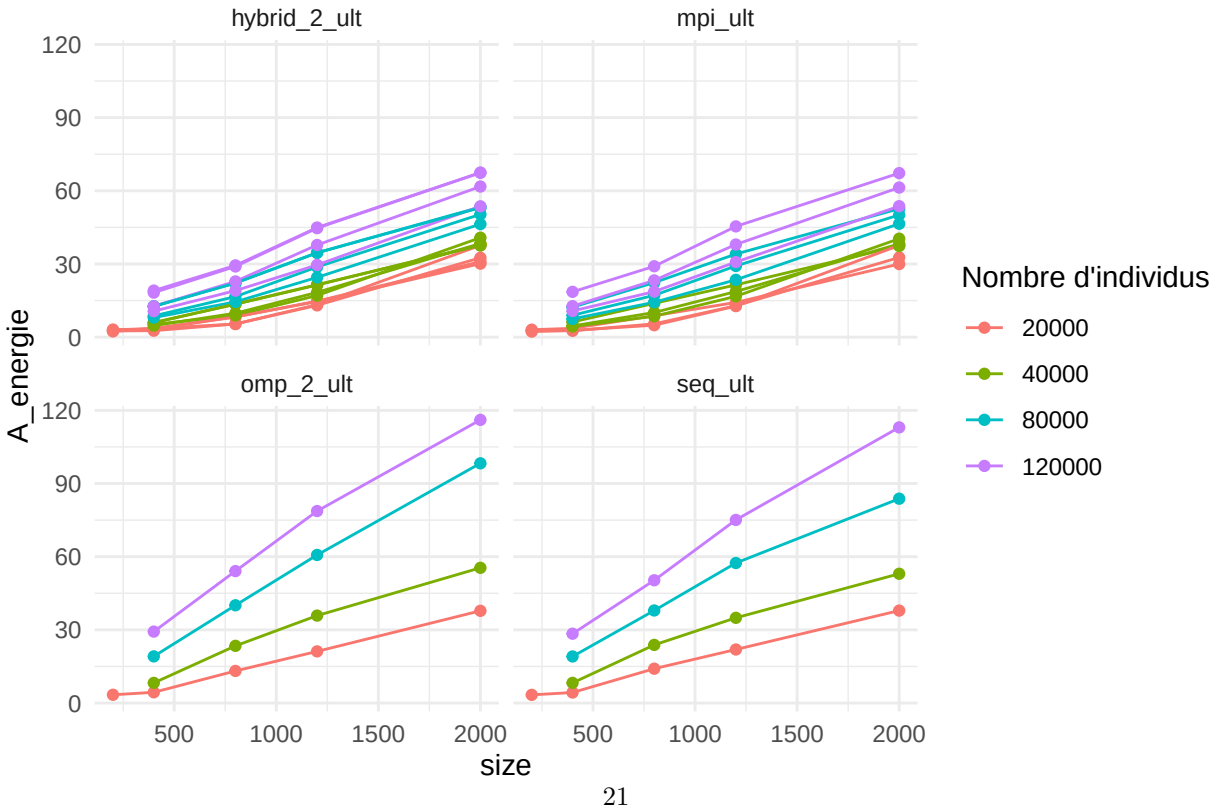


Table 8: Résumé des ratios par algorithme

exec	speedup	energy_ratio	n
hybrid_2_ult	4.780900	0.5921708	64
mpi_ult	5.161662	0.5660440	48
omp_2_ult	0.989963	1.0276055	16
seq_ult	1.000000	1.0000000	16

On peut maintenant comparer les quatre versions principales. En observant d'abord l'énergie, on remarque que les deux versions utilisant MPI sont très proches l'une de l'autre, tout comme la version séquentielle et la version OpenMP, qui présentent des courbes quasiment identiques. Entre ces deux groupes, l'écart peut atteindre un facteur 2 (par exemple 52 J contre 113 J), ce qui représente un gain énergétique très significatif.

En examinant ensuite le temps d'exécution, on retrouve exactement la même séparation en deux groupes. Cette fois, l'écart est encore plus marqué : les versions basées sur MPI atteignent un speedup d'environ 4 dans leur configuration optimale, c'est-à-dire lorsque les 16 cœurs sont pleinement exploités.

L'analyse moyenne confirme ces observations : la méthode OpenMP n'apporte pratiquement aucun bénéfice par rapport à la version séquentielle. En revanche, la version MPI offre un gain très net, à la fois en temps et en énergie, avec un speedup supérieur à 5 et une consommation énergétique réduite d'environ 44 %. Pour ce problème, la méthode MPI apparaît donc clairement comme la version la plus performante et la plus efficace énergétiquement.

Table 9: Résumé des ratios par algorithme (config optimale)

exec	speedup	energy_ratio	n
hybrid_2_ult	6.211551	0.5009782	16
mpi_ult	6.312233	0.4924180	16
omp_2_ult	0.989963	1.0276055	16
seq_ult	1.000000	1.0000000	16

5.4 Conclusion finale du modèle SEIR

L'ensemble des expérimentations montre que le modèle SEIR se prête très bien au parallélisme inter-processus (MPI), mais très mal au parallélisme intra-nœud (OpenMP) et à la vectorisation. Les dépendances locales, les accès mémoire irréguliers et la structure même du modèle limitent fortement les gains possibles via les optimisations classiques.

Les optimisations du compilateur confirment rapidement leurs limites : O3 s'impose systématiquement comme le meilleur compromis, tandis que O0 dégrade fortement les performances. La vectorisation automatique n'apporte qu'un bénéfice marginal, et la vectorisation forcée, bien que parfois plus sobre énergétiquement, reste légèrement plus lente. De manière générale, la vectorisation n'est pas un levier déterminant pour ce modèle.

Les optimisations liées à la taille des structures produisent des effets modestes : la version cache-friendly réduit légèrement le temps d'exécution, mais l'impact énergétique reste faible et souvent noyé dans la variabilité des mesures. Le modèle SEIR n'est donc pas particulièrement sensible à la localité mémoire.

Les approches OpenMP confirment cette tendance : la version naïve souffre de synchronisations et d'allocations excessives, tandis que la version propre (omp_v2) reste limitée par les dépendances du modèle. Au final, OpenMP n'apporte aucun gain significatif, ni en temps ni en énergie.

À l'inverse, MPI se révèle nettement plus efficace. Le découpage spatial du domaine, associé à des communications limitées aux frontières, permet une montée en charge solide jusqu'à 12 processus, avec un optimum

énergétique et temporel autour de cette valeur. Même à 16 processus, malgré un léger overhead, MPI reste très performant. Dans sa configuration optimale, la version MPI pure atteint un speedup d'environ $\times 6$ et réduit la consommation d'énergie d'environ 50 %, ce qui en fait la solution la plus rapide, la plus stable et la plus économe pour ce modèle.

Les versions hybrides (MPI + OpenMP) montrent un comportement intermédiaire : la version propre (hybrid_v2) est plus rapide mais plus énergivore, car elle sollicite fortement les unités vectorielles. Malgré quelques configurations intéressantes (notamment 8 MPI / 2 OMP), l'hybride ne parvient pas à surpasser MPI pur, ni en temps ni en énergie.

En conclusion, le modèle SEIR bénéficie très peu des optimisations locales (cache, vectorisation, OpenMP), mais exploite pleinement le parallélisme inter-processus. La version MPI pure, dans sa configuration optimale, constitue donc la solution la plus performante et la plus efficace énergétiquement pour ce type de simulation

6 Jeu De La Vie

6.1 Présentation

Le Jeu de la vie de Conway est un automate cellulaire où chaque cellule est vivante ou morte, et évolue à chaque itération selon un simple comptage de voisines. Ce modèle, régulier et massivement parallélisable, se prête bien à l'expérimentation informatique et à l'évaluation de différentes optimisations.

Dans cette étude, nous analysons l'impact :

- des optimisations du compilateur (O0–Ofast),
- du blocking (tailles 4 à 256),
- de la vectorisation (par défaut, NV, VF),
- du cœur de calcul (C1, C2),
- de la représentation matricielle (2D ou linéarisée),
- et des paradigmes séquentiel, OpenMP et MPI.

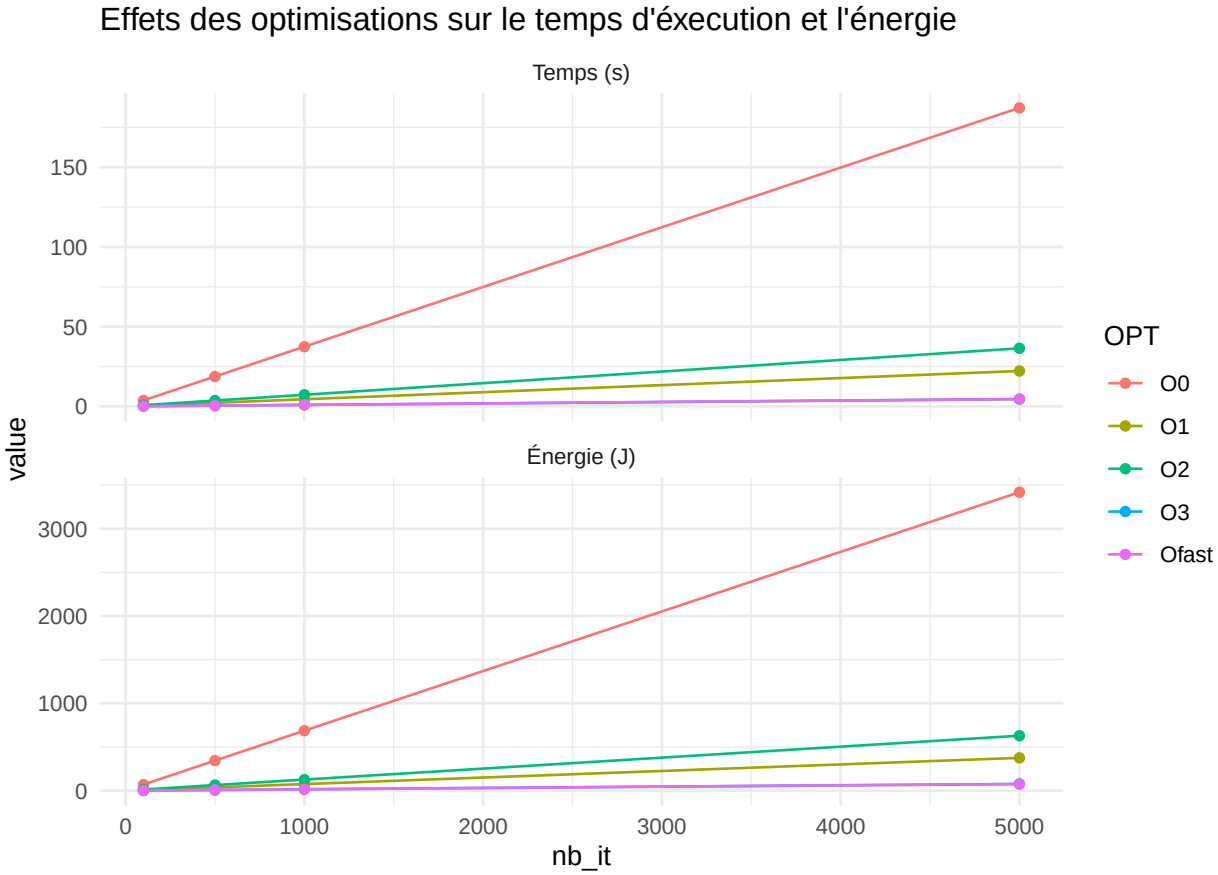
Chaque configuration est identifiée par un code (ex. seq_lin_blk_NV_C1_B2) indiquant le paradigme, la représentation, le blocking, la vectorisation, le cœur utilisé et la campagne de tests.

Les algorithmes reposent tous sur la même logique : deux grilles (état courant / état suivant) et un calcul de voisinage pour chaque cellule. OpenMP parallélise ce calcul via des directives, tandis que MPI découpe la grille entre processus et échange des halos à chaque itération.

6.2 Analyse

6.2.1 Analyse de l'algorithme sequentiel

6.2.1.1 Analyse des optimisations



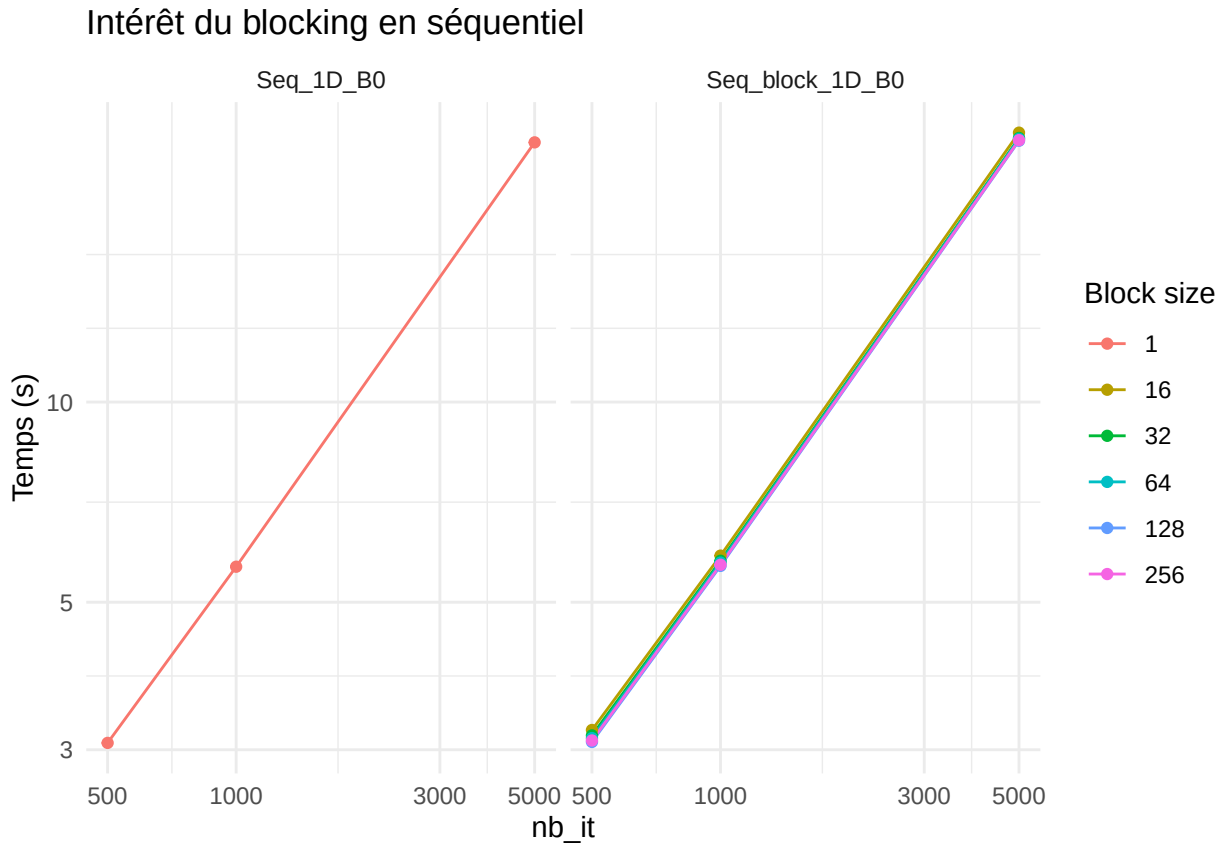
On observe ici que la version compilée en O0 est, une fois encore, nettement moins performante que les trois autres optimisations. L'évolution de l'énergie consommée est parfaitement corrélée au temps d'exécution, et ce pour toutes les optimisations et toutes les tailles, ce qui est un point positif. On remarque également que les méthodes d'optimisation se classent clairement : O0 » O1 > O2 > O3 = Ofast.

Les écarts de temps et d'énergie restent importants entre O1/O2 et O3/Ofast. Les versions O3 et Ofast sont strictement équivalentes, ci bien qu'elles ce supperpose parfaitement, tant en temps qu'en énergie. Les différences observées se situent dans la marge d'erreur.

Rien qu'en changeant la méthode d'optimisation, on peut atteindre un speedup allant jusqu'à 41, avec une économie d'énergie du même ordre. Entre O1 et O3, on obtient déjà un speedup d'environ 8, accompagné d'un gain énergétique équivalent.

Enfin, on constate que la puissance reste globalement constante, quelle que soit la taille ou l'optimisation, ce qui explique la forte corrélation entre temps et énergie. On note seulement une légère différence : environ 18 W pour O0, contre 16.5 à 17.3 W pour les autres optimisations.

6.2.1.2 Analyse de l'impact du blocking:



On observe que la taille du bloc a un impact très limité : les courbes restent quasiment superposées et le blocking n'apporte qu'un léger gain d'environ 3 %, avec une valeur optimale autour de 128. La consommation énergétique suit exactement la même tendance que le temps d'exécution, ce qui confirme la cohérence des mesures.

En pratique, comparée à la version sans blocking, la taille des blocs n'influence ni la rapidité du programme ni l'énergie consommée de manière significative. Les écarts mesurés restent dans la marge d'erreur, ce qui ne permet pas de conclure à un effet réel du blocking dans ce cas précis.

6.2.1.2 Coeur de calcul C1 vs C2 sur sequentielle blocké

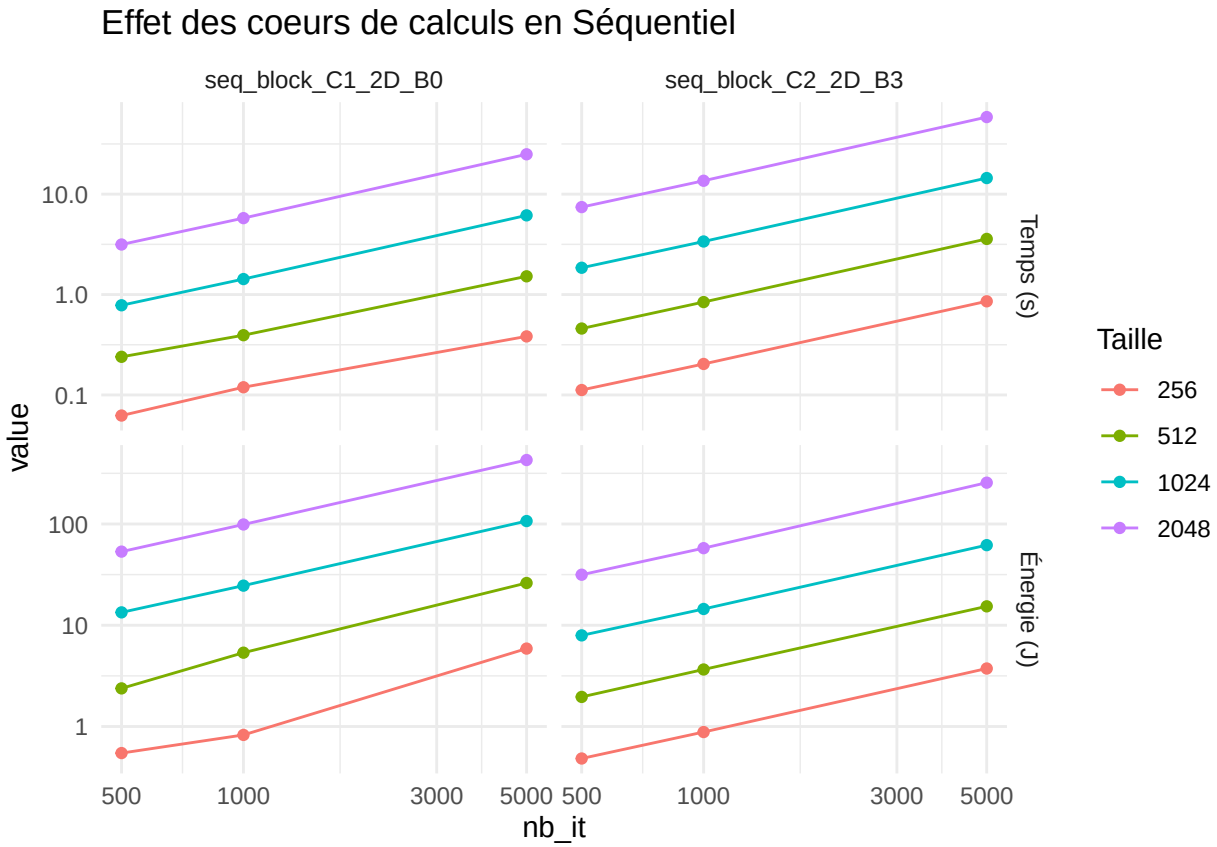


Table 10: Résumé des ratios temps/énergie pour seq_block_C2_2D vs seq_block_C1_2D

exec	mean_speedup	mean_energy_ratio	n
seq_block_B0	2.190912	1.517868	12
seq_block_C2_2D_B3	1.000000	1.000000	12

On observe deux comportements très différents entre les deux cœurs. Sur le temps d'exécution, C1 est nettement plus rapide : en moyenne, il obtient un speedup d'environ 2,4× par rapport à C2. En revanche, du point de vue énergétique, la tendance s'inverse complètement : C2 consomme environ 40 % d'énergie en moins que C1 pour une charge de travail identique.

Cela montre que le choix du cœur de calcul a un impact majeur sur les performances et sur l'efficacité énergétique.

Cette différence peut s'expliquer par la manière dont les unités vectorielles sont exploitées. Le compilateur semble parvenir à vectoriser plus agressivement sur C1, ce qui réduit le temps d'exécution. Cependant, cette vectorisation n'utilise probablement pas pleinement la largeur des registres SIMD : les vecteurs sont partiellement remplis, ce qui signifie que l'unité vectorielle effectue des opérations sur des lanes "vides". Dans ce cas, on gagne en vitesse parce que l'opération vectorielle traite plusieurs éléments à la fois, mais on perd en efficacité énergétique : l'unité SIMD consomme de l'énergie pour des lanes inutilisées. Autrement dit, C1 exécute plus vite, mais gaspille une partie de son énergie dans des opérations qui ne contribuent pas réellement au calcul.

À l'inverse, C2 est moins performant en temps, mais plus efficace énergétiquement, probablement parce que sa vectorisation est moins agressive ou mieux adaptée à la structure des données, ce qui limite le gaspillage énergétique dans les unités SIMD.

6.2.1.3 Cœurs sur la version séquentielle non bloquée :

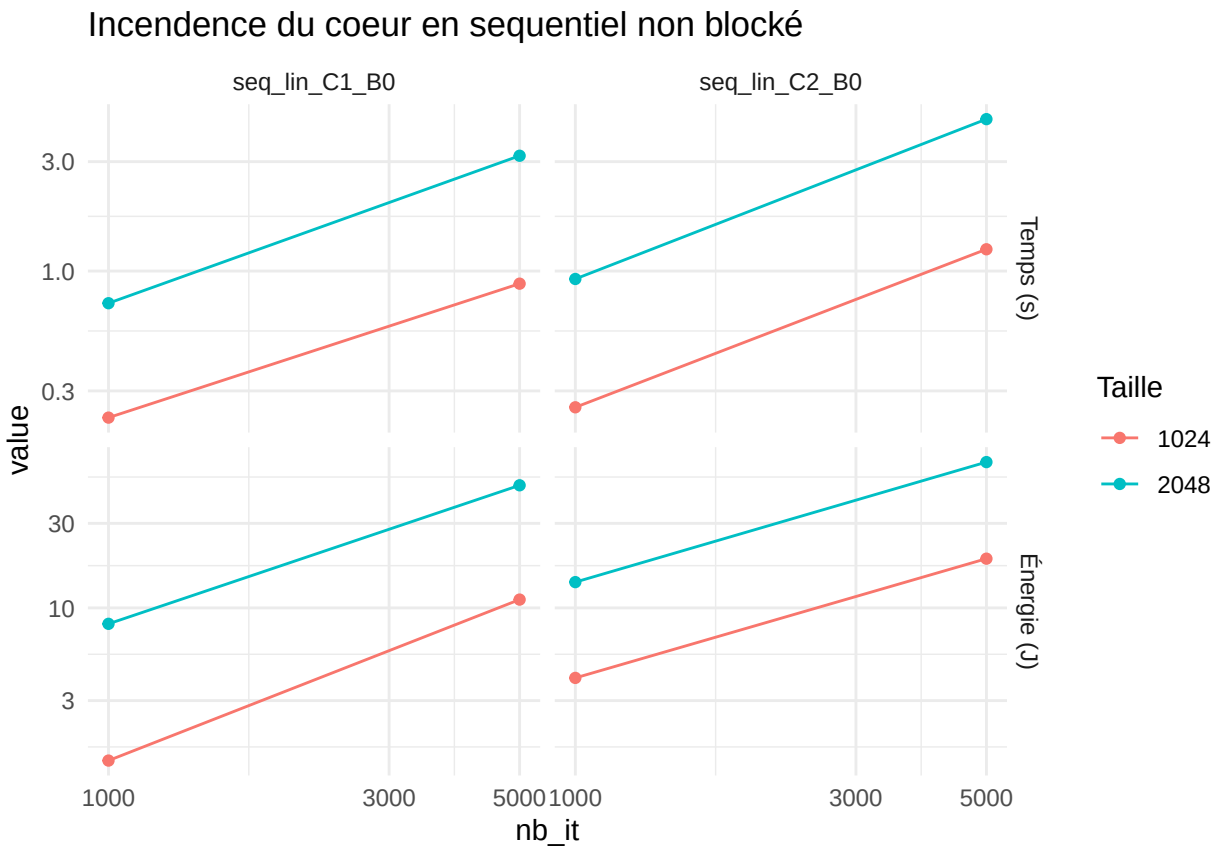


Table 11: Résumé des ratios temps/énergie pour seq_lin_C2_B0 vs seq_lin_C1_B0

exec	mean_speedup	mean_energy_ratio	n
seq_lin_C1_B0	1.310505	0.5626562	4
seq_lin_C2_B0	1.000000	1.0000000	4

Ici, on remarque que le cœur C1 est plus intéressant : il est environ 31 % plus rapide, avec un gain énergétique variable pouvant atteindre 44 %, et dans certains cas une consommation jusqu'à trois fois plus faible.

Dans cette configuration, C1 apparaît donc nettement meilleur, à la fois plus rapide et plus économe. Cela suggère que la vectorisation y est plus efficace : les registres SIMD semblent mieux remplis, ce qui permet d'obtenir un gain simultané en performance et en énergie.

6.2.1.4 Vectorisation en sequentiel

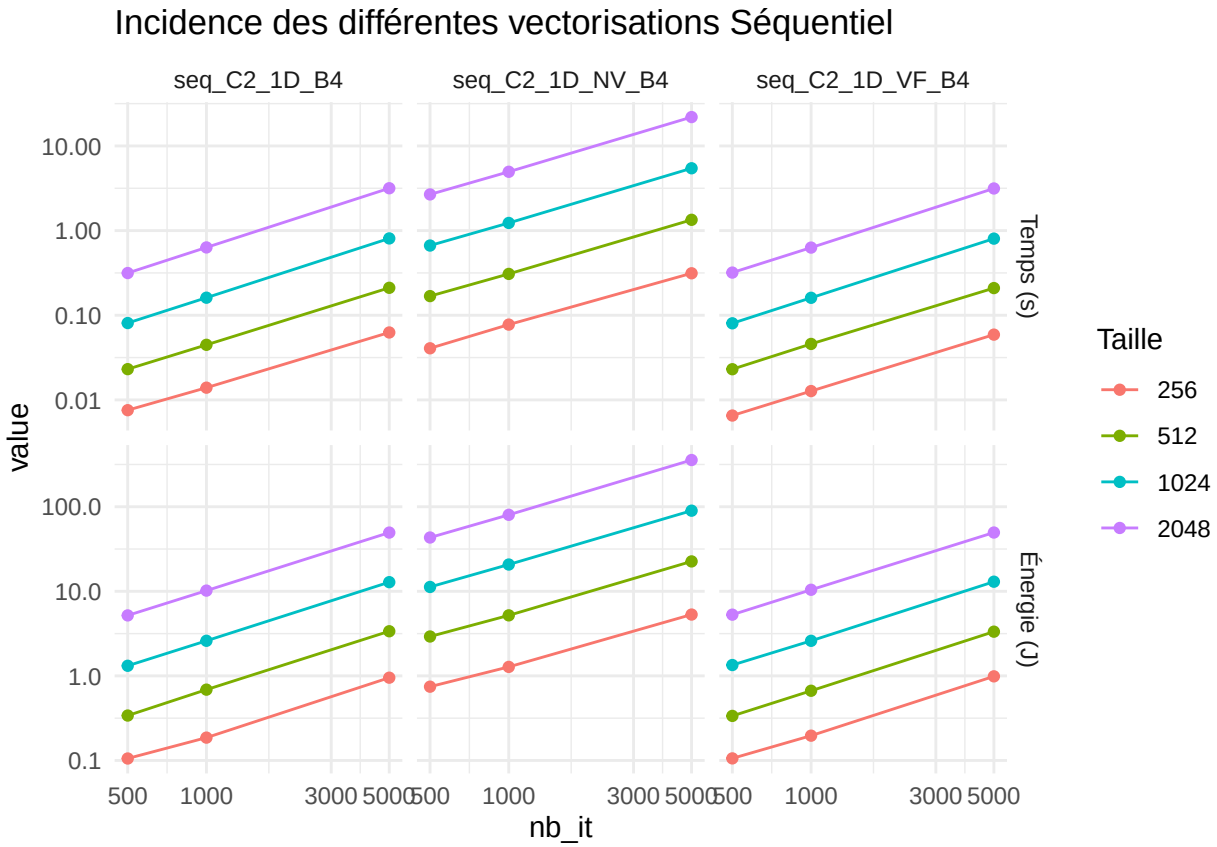


Table 12: Résumé des ratios temps/énergie

exec	mean_speedup	mean_energy_ratio	n
seq_C2_1D_B4	1.0000000	1.000000	12
seq_C2_1D_NV_B4	0.1496416	7.443525	12
seq_C2_1D_VF_B4	1.0262129	1.009941	12

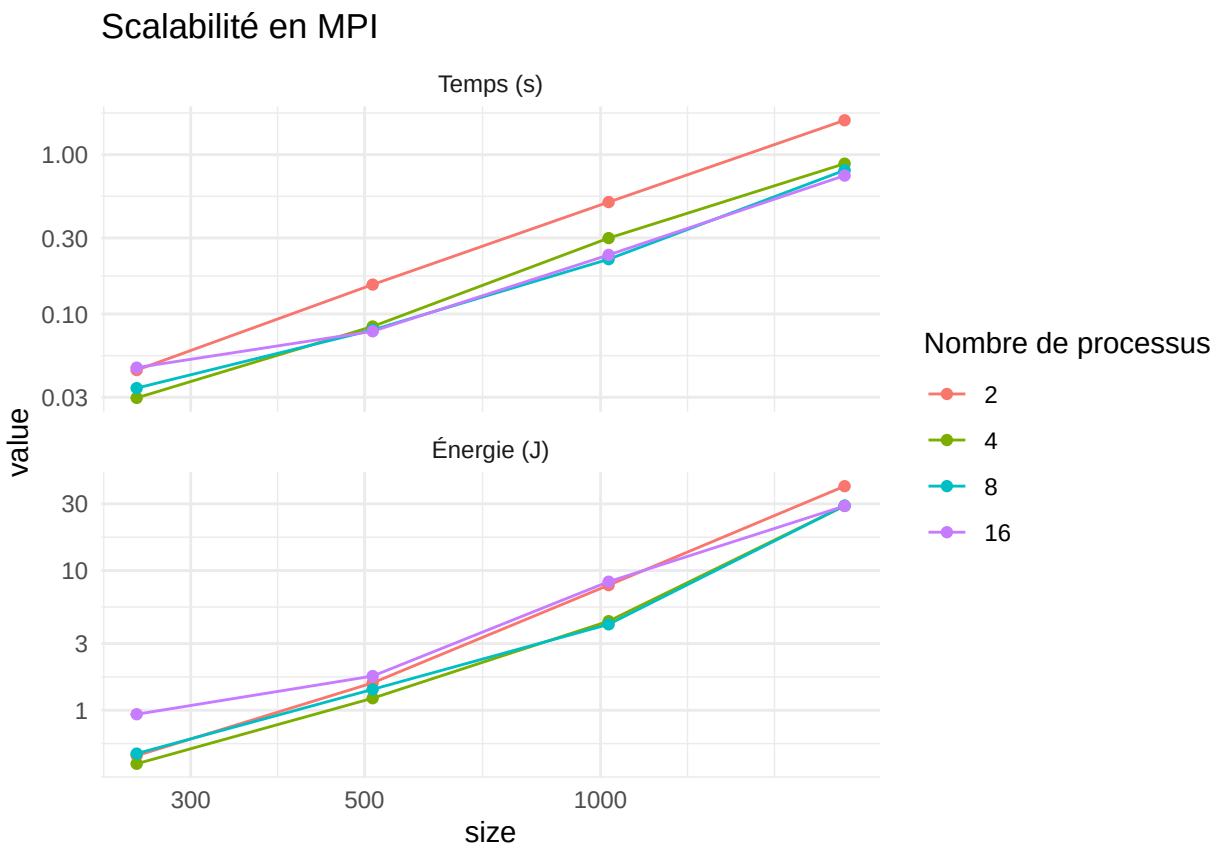
Table 13: Consommation totale et moyenne d'énergie par algorithme

exec	total_energy	mean_energy	n
seq_C2_1D_B4	87.25805	7.271504	12
seq_C2_1D_NV_B4	640.39898	53.366582	12
seq_C2_1D_VF_B4	87.82674	7.318895	12

En séquentiel, on retrouve la même tendance qu'en MPI : la version non vectorisée est nettement derrière. En revanche, VF et l'auto-vectorisation donnent pratiquement les mêmes performances. Comme la matrice n'est pas découpée entre plusieurs processus, les boucles sont longues et régulières, ce qui permet au compilateur de vectoriser efficacement tout seul. Résultat : forcer la vectorisation n'apporte rien, mais ne pénalise pas non plus.

6.2.2 Paradigme MPI :

6.2.2.1 Scalabilité

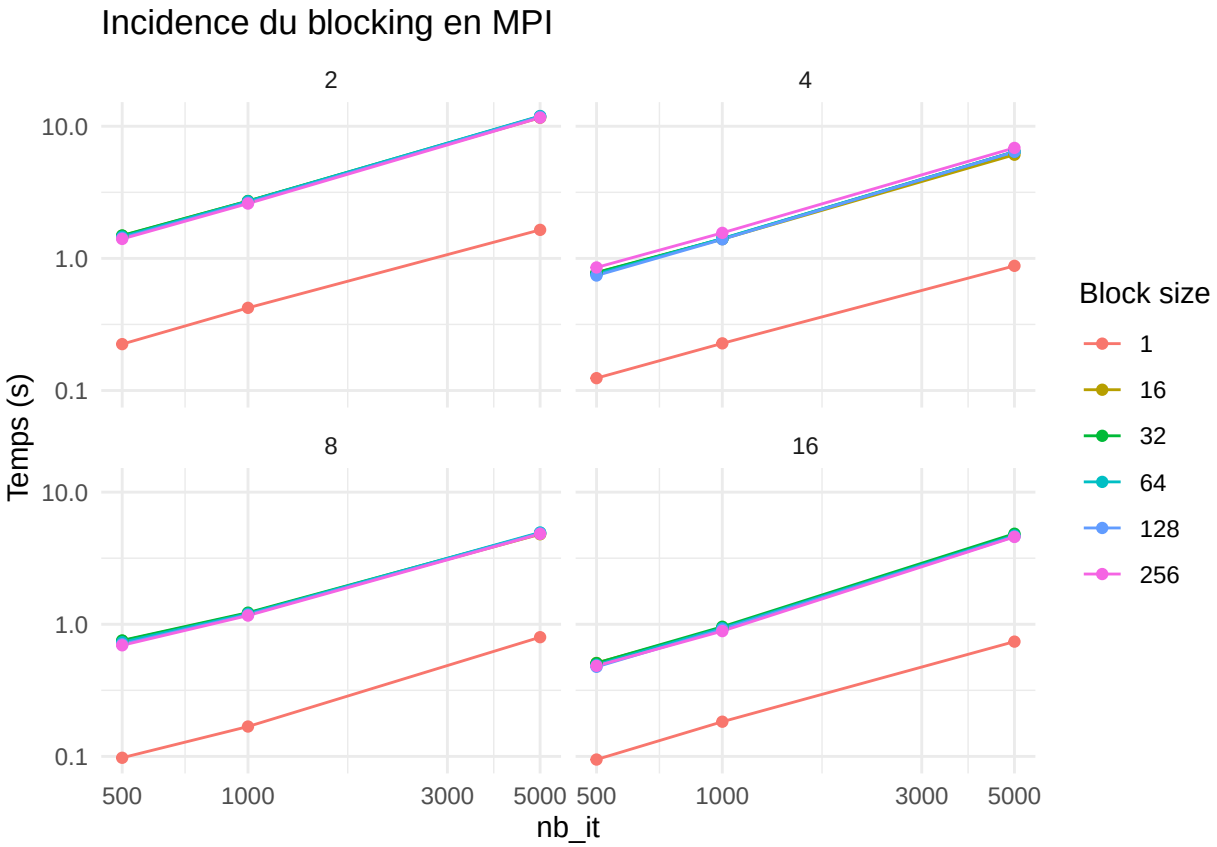


La scalabilité avec le nombre de processus est réelle mais reste modérée : passer de 2 à 16 processus ne divise le temps d'exécution que par deux, et réduit la consommation énergétique d'environ 30 %. Autrement dit, augmenter le nombre de processus apporte un gain, mais ce gain n'est pas proportionnel au parallélisme engagé.

Sur le plan énergétique, un nombre élevé de processus devient avantageux lorsque le nombre d'itérations augmente. Cela s'explique probablement par une réduction de la charge par processus, limitant le throttling et permettant aux cœurs de fonctionner de manière plus stable et plus efficace.

Lorsque la taille du problème augmente, on observe un seuil à partir duquel l'ajout de processus devient réellement bénéfique. En dessous de ce seuil, le programme est non seulement plus lent, mais aussi plus énergivore. Au-delà, l'exécution à 16 processus devient clairement avantageuse en temps, même si la consommation énergétique reste plus élevée que pour des configurations avec moins de processus. Cette surconsommation est très probablement liée au surcoût du modèle MPI : davantage de communications, davantage de synchronisations, donc davantage d'énergie dépensée.

6.2.2.2 Incidence du blocking par rapport à la version non bloquée :



La taille des blocs n'apporte pratiquement aucun bénéfice dans la version MPI : les performances des différentes configurations restent très proches, et les variations observées (par exemple avec des blocs de 256) sont faibles et non significatives. Selon le nombre de processus, un même bloc peut même devenir la configuration la plus lente, ce qui confirme l'absence d'effet réel du choix de la taille.

En revanche, l'activation du blocking elle-même entraîne une dégradation majeure : la version bloquée est environ 7× plus lente que la version sans blocking, et consomme près de 4× plus d'énergie. Le blocking pénalise donc simultanément le temps d'exécution et l'efficacité énergétique.

Ainsi, dans le Jeu de la vie, le blocking représente une perte nette de performance, en MPI comme en séquentiel. Dans l'approche utilisée ici, il doit être évité, car il dégrade fortement le calcul sans apporter de bénéfice mesurable.

6.2.2.3 Coeurs de calculs en MPI :

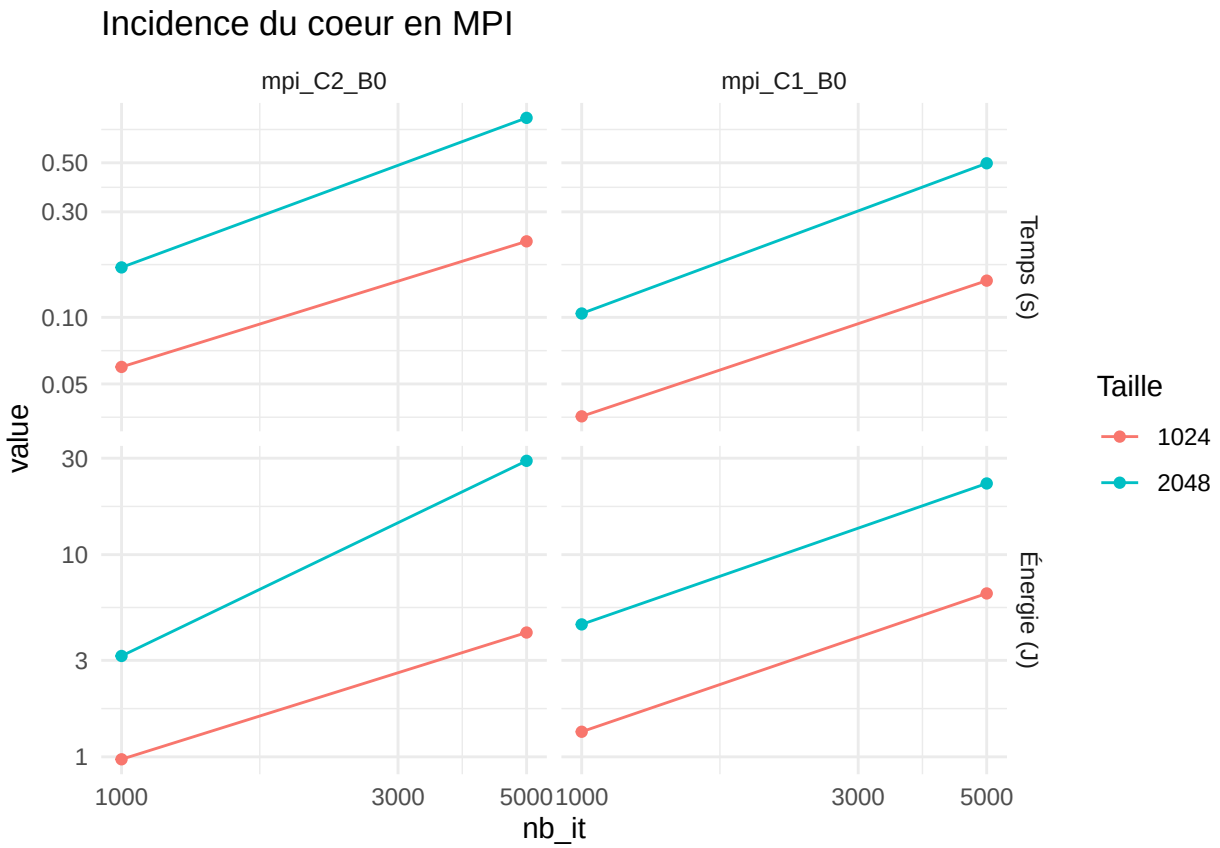


Table 14: Résumé des ratios temps/énergie pour mpi_C1_B0 vs mpi_C2_B0

exec	mean_speedup	mean_energy_ratio	n
mpi_B0	1.000000	1.000000	4
mpi_C1_B0	1.599852	1.283662	4

Avec MPI, le cœur C1 reste systématiquement le plus rapide. En revanche, il devient plus énergivore que C2. La division du domaine entre processus réduit la taille des blocs traités localement, ce qui rend la vectorisation moins efficace : les unités SIMD de C1 sont moins bien remplies, ce qui entraîne un surcoût énergétique. Ce phénomène n'apparaissait pas en séquentiel, où chaque cœur traitait des blocs beaucoup plus grands.

6.2.2.4 Synthèse générale sur les coeurs :

Le cœur C1 est globalement le plus performant en temps, mais son efficacité énergétique dépend fortement de la taille des blocs de données et du mode d'exécution :

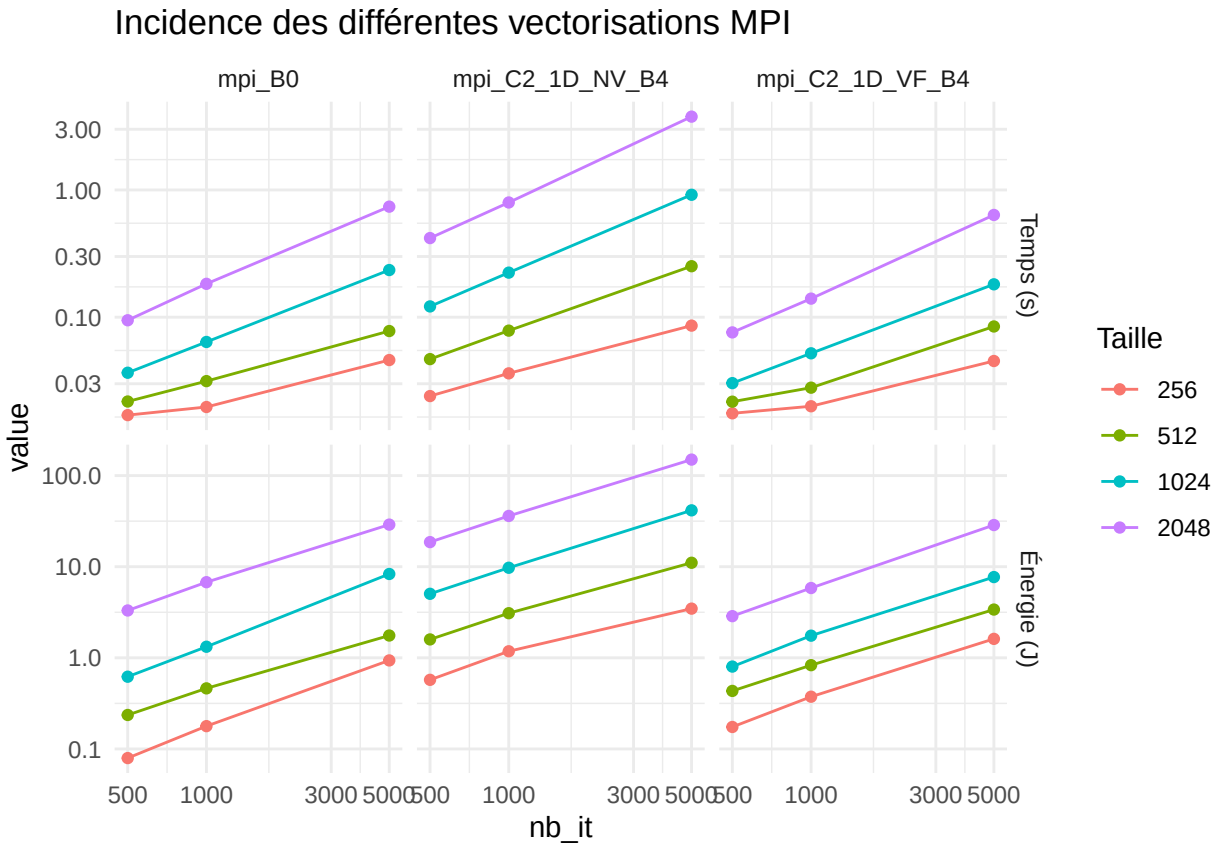
Blocs grands (séquentiel non bloqué) → C1 est le meilleur en temps et en énergie.

Blocs petits ou fragmentés (MPI, séquentiel bloqué) → la vectorisation devient moins efficace, C1 gaspille de l'énergie, et C2 devient plus intéressant.

En résumé, C1 excelle lorsque la vectorisation peut s'exprimer pleinement, tandis que C2 est préférable lorsque la charge est fragmentée ou défavorable à une exploitation optimale des unités SIMD.

Un point notable ressort : C1 se montre régulièrement plus performant que C2, alors que C2 était utilisé par défaut dans les benchmarks. Cela suggère que le choix initial du cœur n'était pas optimal. Une nouvelle campagne de tests centrée sur C1 aurait été pertinente, mais n'a pas pu être réalisée faute de temps.

6.2.2.5 Vectorisation sur MPI



(cf table 6, 7 et 8) La vectorisation MPI a un impact clair. L'auto-vectorisation (O3) apporte déjà un gain constant par rapport à la version non vectorisée (NV), quelle que soit la taille. La vectorisation forcée (VF), elle, ne change rien sur les petites matrices — les écarts sont dans la marge d'erreur — mais devient réellement efficace sur les grandes tailles, avec un gain net de 15 à 20 %.

Côté énergie, la version NV est de loin la plus coûteuse : 5 à 6 fois plus que les versions vectorisées, indépendamment de la taille. Pour VF, le comportement est plus contrasté : elle consomme près du double sur les petites tailles, mais devient légèrement plus économe sur les grandes matrices (environ –15 %). Au total, si l'on somme l'énergie sur toutes les tailles, VF finit par être légèrement meilleure, et cet avantage augmente avec la taille des matrices.

En bref : la vectorisation devient intéressante — en performance comme en énergie — dès que les matrices deviennent grandes.

6.2.3 Paradigme OpenMP :

6.2.3.1 Blocking

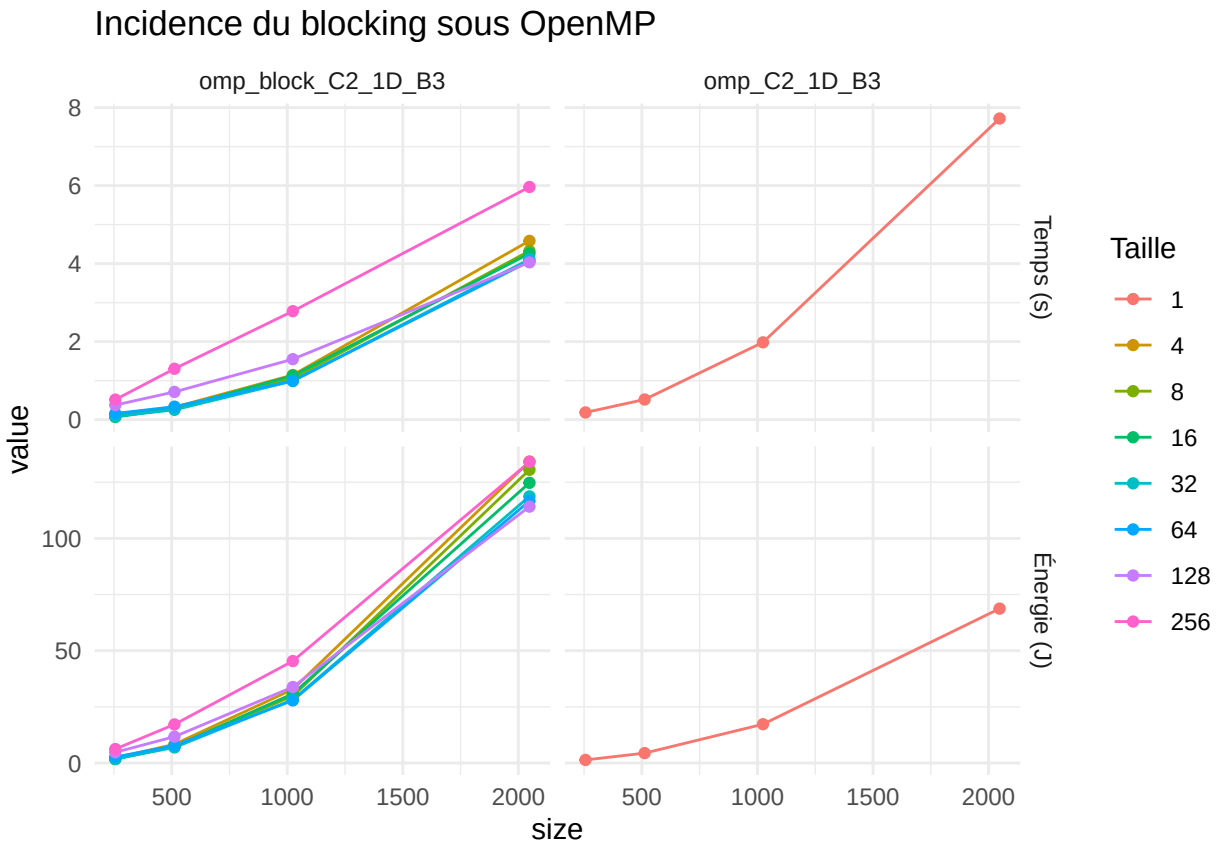


Table 15: Résumé des ratios temps/énergie pour omp_block_C2_1D_B3 vs omp_C2_1D_B3

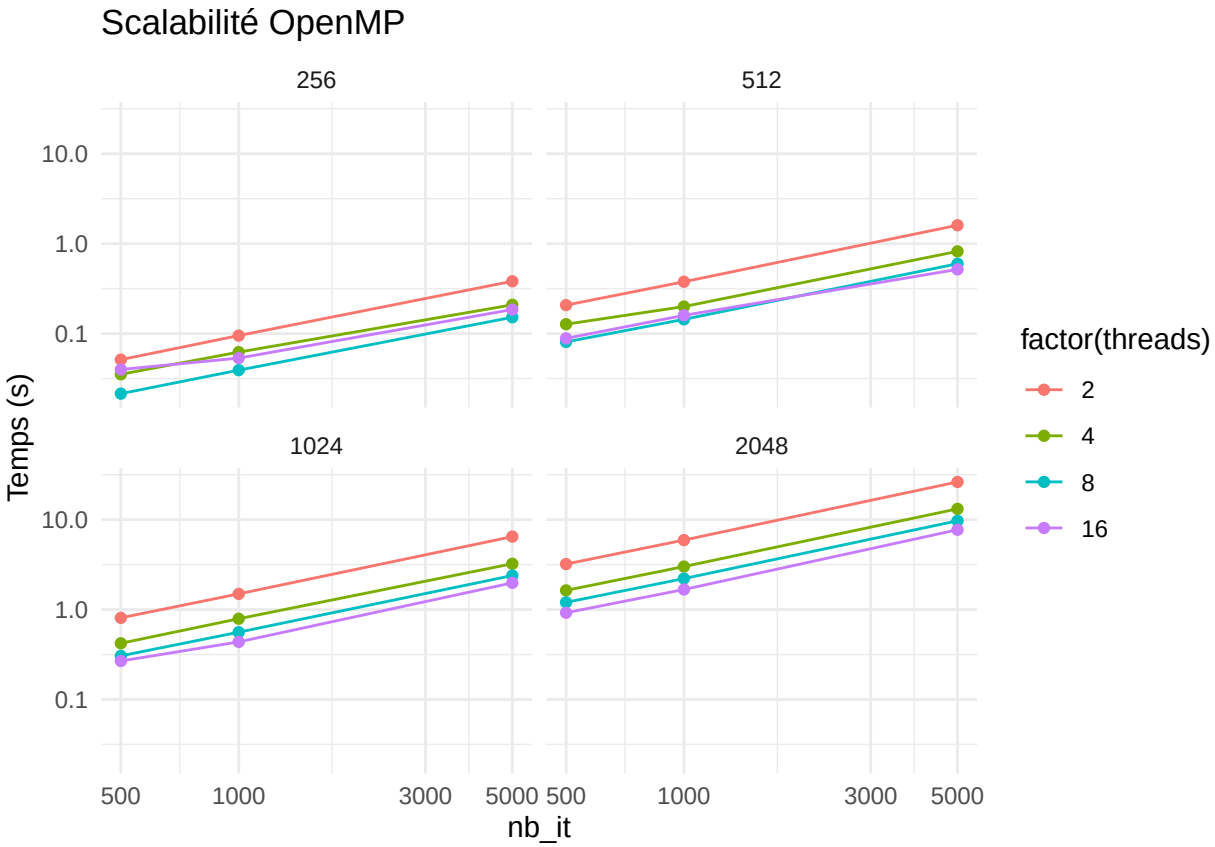
exec	mean_speedup	mean_energy_ratio	n
omp_block_C2_1D_B3	2.165747	1.530775	4
omp_C2_1D_B3	1.000000	1.000000	4

L'analyse du blocking sur OMP nous offre une perspective totalement nouvelle sur l'intérêt de cette technique. On observe ici que le blocking apporte un gain en temps, avec un speedup pouvant atteindre 2, mais au prix d'une surconsommation d'énergie pouvant aller jusqu'à 1,5× dans la configuration la plus rapide.

Cela s'explique par le découpage de la grille pour l'utilisation d'OpenMP : avec le blocking, chaque thread dispose de son propre bloc à calculer. Une fois ce bloc terminé, il passe au suivant. Ce fonctionnement réduit fortement les collisions d'accès mémoire entre threads, leur évitant de perdre du temps à attendre qu'un autre ait terminé sa lecture. Le découpage est donc très efficace pour éviter les conflits entre threads et améliorer la localité mémoire.

Cependant, on constate également une augmentation de la consommation d'énergie. Cette hausse est probablement liée à une utilisation plus intensive des unités de calcul et à une pression mémoire accrue, conséquence directe de la meilleure exploitation du parallélisme.

6.2.3.2 Scalibilité en OpenMP



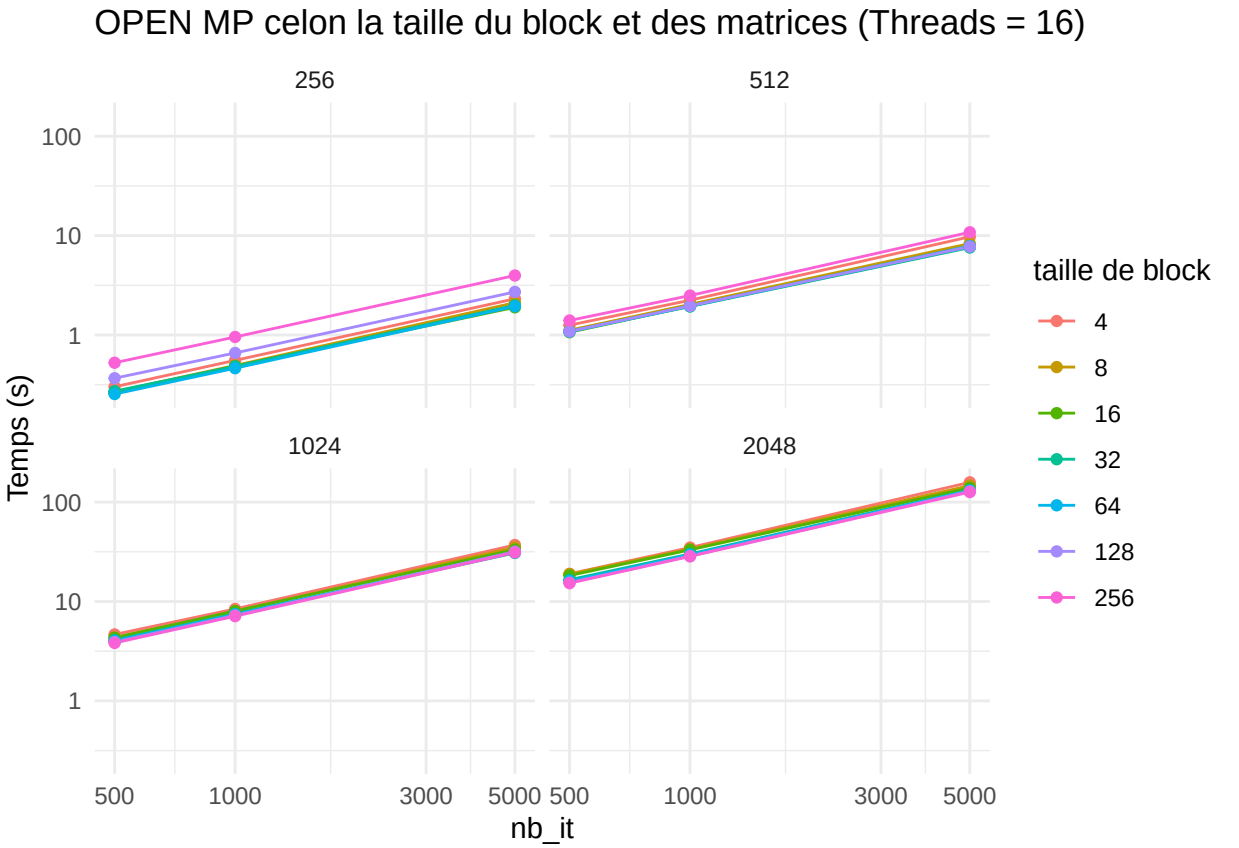
On observe ici une tendance très nette : le scaling OpenMP est excellent dès que la taille de la grille et le nombre d'itérations deviennent suffisants. Dans chaque configuration où la charge de travail est assez élevée, l'augmentation du nombre de threads se traduit systématiquement par une baisse du temps d'exécution, ce qui confirme une bonne exploitation du parallélisme.

Les seules situations où cette scalabilité n'est pas vérifiée concernent les très petites tailles (256, 512) combinées à un faible nombre d'itérations (<1000). Dans ces cas, le travail par thread est trop faible pour amortir les coûts de synchronisation, ce qui explique que 16 threads n'apportent aucun gain, voire dégradent légèrement les performances.

En dehors de ces cas marginaux, la scalabilité est bien présente et cohérente, avec un comportement qui s'améliore clairement lorsque la charge de calcul augmente.

Quant à la consommation d'énergie, les courbes se superposent parfaitement avec celles du temps.

6.2.3.3 Blocking dans OpenMP



(Le graphique est en échele logarithmique)

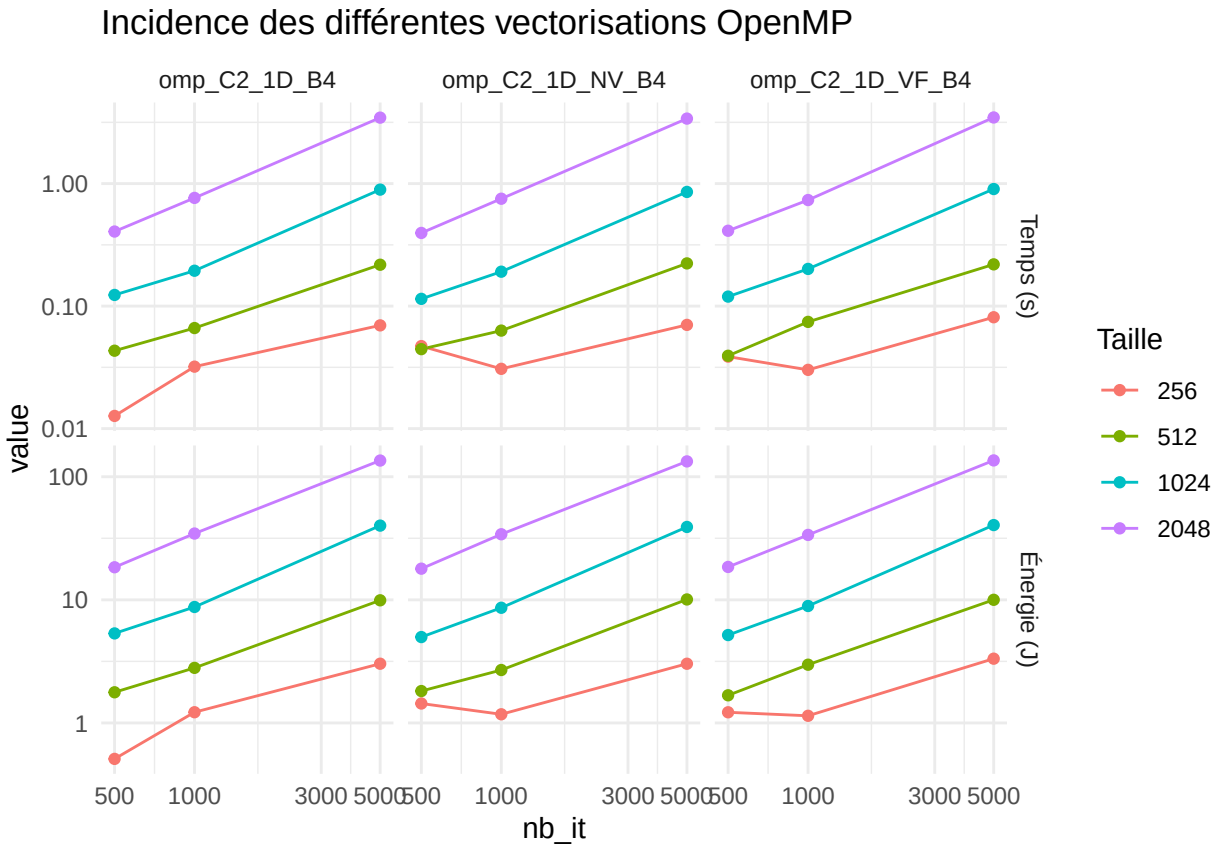
En OpenMP, la taille du bloc a une influence marquée sur les performances : selon les configurations, on peut observer des variations de temps d'exécution allant d'un facteur 2 à 3 entre les tailles extrêmes. Un phénomène intéressant apparaît alors : la taille de bloc optimale semble évoluer de manière quasi linéaire avec la taille de la matrice. Pour une grille de 256, le bloc idéal est autour de 16 ; pour 512, il passe à 32 ; et ainsi de suite. On peut donc extrapoler qu'un bloc de 256 serait probablement optimal pour une matrice de 4096.

Ce comportement s'explique assez naturellement : avec 16 threads, un bloc de taille 16 divise la matrice en autant de sous-blocs que de threads, ce qui permet une distribution simple et efficace du travail. Chaque thread reçoit un bloc unique et peut ensuite travailler sans synchronisation supplémentaire. Rien de surprenant donc : le blocking s'aligne sur le nombre de threads pour maximiser la régularité du travail.

En réduisant le nombre de threads, on observe que cette relation se maintient. Avec 8 threads, la taille de bloc optimale double, ce qui est cohérent avec la réduction du parallélisme disponible : un bloc de 64 devient idéal pour une matrice de 256. Avec seulement 2 threads, la meilleure taille de bloc se stabilise autour de 16, quelle que soit la taille de la matrice. Et avec 4 threads, le bloc 256 devient rapidement avantageux.

Du côté énergétique, la tendance est plus nuancée. Il arrive qu'une taille de bloc légèrement inférieure à celle optimale pour le temps d'exécution soit plus intéressante en termes de consommation. Malgré ces variations, la tendance générale reste la même : plus la matrice est grande, plus la taille de bloc optimale tend à augmenter.

6.2.3.4 Vectorisation en OpenMP



(cf table 10 et 11) Dans le cas d'OpenMP, les différentes méthodes de vectorisation produisent des résultats très proches les unes des autres. Les écarts observés restent faibles et varient légèrement selon le nombre de threads, mais sans jamais dépasser une marge d'erreur raisonnable. Les versions NV et VF suivent pratiquement la même évolution pour 4, 8 et 16 threads, ce qui suggère que la vectorisation n'a pas d'impact significatif dans ce contexte. Il est probable que le compilateur parvienne déjà à vectoriser efficacement le code en OpenMP, rendant les variantes NV et VF presque équivalentes à la version de base, et que les petites fluctuations mesurées soient simplement dues à la variabilité naturelle du runtime plutôt qu'à un effet réel de la vectorisation.

Resume global sur la vectorisation : Au vu de l'ensemble des résultats, la vectorisation apparaît comme une optimisation très efficace dans la majorité des cas, en particulier dès que le calcul est suffisamment structuré ou que la taille des données devient importante. Dans MPI, elle apporte des gains nets en performance et en énergie, et ces gains augmentent avec la taille des matrices. En OpenMP, son impact est plus discret mais reste globalement positif ou neutre, ce qui montre que le compilateur parvient déjà à vectoriser correctement le code.

La seule situation où la vectorisation n'a pas d'effet notable est le séquentiel, où l'auto-vectorisation du compilateur suffit à atteindre les meilleures performances possibles. Dans ce cas, forcer la vectorisation ne change pratiquement rien, car les boucles sont longues, régulières et faciles à optimiser automatiquement.

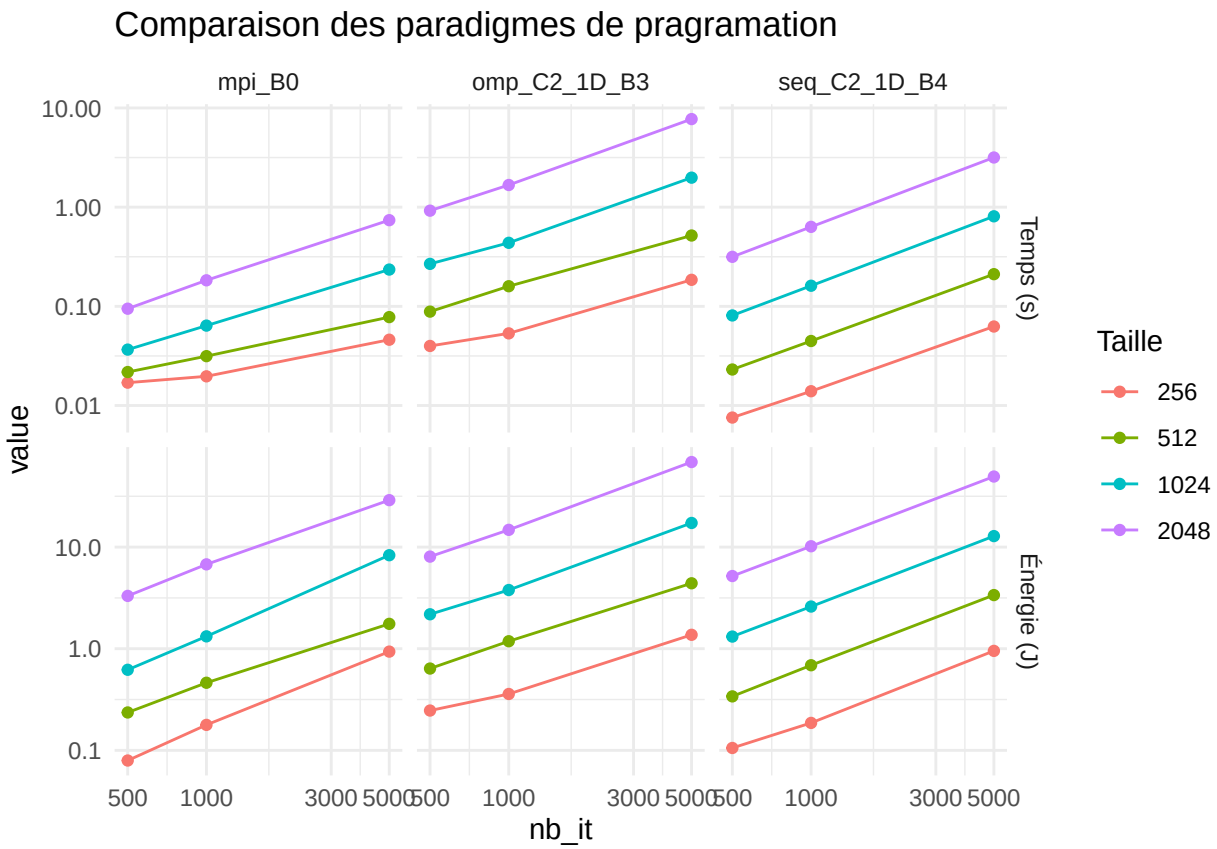
En résumé, la vectorisation est globalement bénéfique, parfois même indispensable (MPI), souvent déjà bien exploitée automatiquement (OpenMP), et seulement marginale en séquentiel où le compilateur fait déjà le travail.

Energétiquement parlant, la vectorisation un gain notable dans la plupart des cas. En MPI, elle réduit massivement la consommation, la version non vectorisée étant de loin la plus coûteuse. En séquentiel,

l'auto-vectorisation du compilateur suffit déjà à obtenir une consommation optimale, et forcer la vectorisation n'apporte rien de plus. En OpenMP, son impact reste faible et largement masqué par le parallélisme, avec des variations minimes entre les différentes versions.

6.3 Comparaison des differents paradigmes

6.3.1 Dans leur version par default



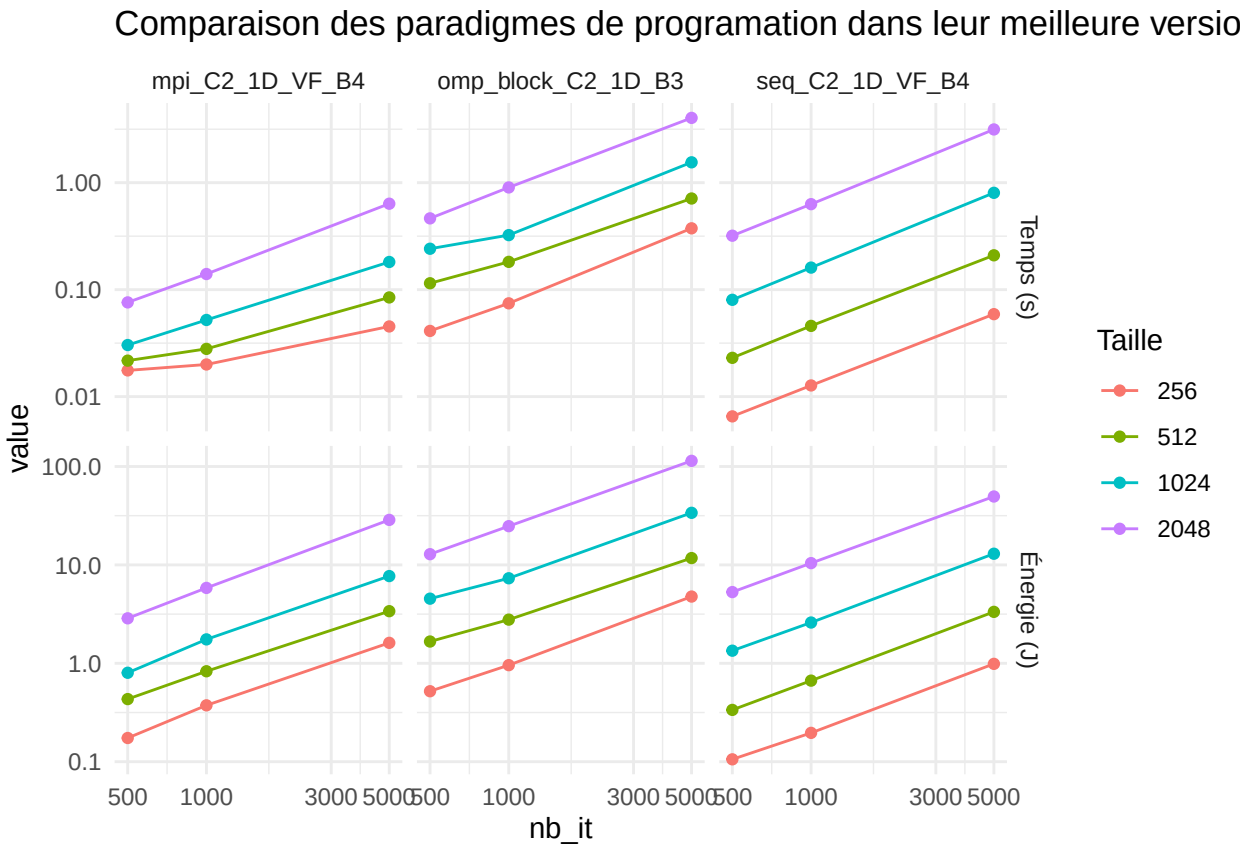
On voit ici que, malgré le fait que les différentes optimisations aient généralement toutes un impact sur les différentes méthodes, si l'on compare chaque méthode entre elles dans leur version la plus simple, les résultats ne sont pas ceux attendus.

L'analyse globale montre que, malgré l'impact réel des différentes optimisations sur chaque méthode de calcul, les performances observées dans leur configuration par défaut ne correspondent pas toujours aux attentes. Dans la configuration initiale — cœur C2, matrice linéaire, sans blocking ni vectorisation explicite — OpenMP se révèle être la méthode la moins performante, aussi bien en temps qu'en énergie. Le temps d'exécution est même deux fois plus long que celui de la version séquentielle, ce qui est particulièrement surprenant. Cette contre-performance peut s'expliquer par la surcouche inhérente à Openmp : gestion des threads, synchronisations, distribution du travail... autant d'éléments qui semblent ici annuler tout gain potentiel. Sur le plan énergétique, la tendance est identique : OpenMP consomme 20 à 30 % de plus que la version séquentielle, ce qui en fait la méthode la plus énergivore dans cette configuration. Ce résultat étant étonnant, il reste possible qu'une erreur d'intégration de ma part soit en cause. C'est un point que j'aurais souhaité approfondir davantage.

À l'inverse, MPI s'impose clairement comme la méthode la plus performante dans cette configuration de

base, avec un speedup d'environ 4× par rapport au séquentiel. Ce gain reste toutefois limité par les coûts de communication : à chaque itération, les processus doivent échanger leurs bordures et se synchroniser, ce qui impose d'attendre le processus le plus lent. Ces échanges, indispensables à la cohérence du calcul, réduisent mécaniquement le speedup maximal théorique.

6.3.2 Dans leur meilleurs configuration :



Ici, la version séquentielle ainsi que MPI ont hérité de la vectorisation forcée, tandis qu'OpenMP a hérité du blocking en taille 128. Il aurait également été intéressant de s'intéresser au cœur C1, qui semblait bien plus performant, mais comme il n'a pas été testé avec OpenMP, je réserve cette variation pour le dernier graphique.

En observant maintenant chaque paradigme dans sa meilleure configuration, les tendances se confirment. La version séquentielle bénéficie légèrement de la vectorisation forcée, MPI en bénéficie fortement, tandis qu'OpenMP tire profit du blocking (ici en taille 128). OpenMP améliore ainsi légèrement ses performances, avec un speedup d'environ 2× par rapport à sa version de base, mais reste malgré tout la méthode la plus lente et la plus énergivore.

MPI, de son côté, creuse encore l'écart et atteint un speedup d'environ 5× face à la version séquentielle optimisée. Sur le plan énergétique, la hiérarchie reste identique : MPI est la méthode la plus économe, tandis qu'OpenMP demeure la moins favorable.

7 Conclusion

7.1 Résumé synthétique

Ce travail met en évidence des tendances robustes et reproductibles sur l'impact des optimisations étudiées — compilation, localité mémoire, vectorisation et parallélisme — à travers trois codes représentatifs : **multiplication matricielle**, **modèle SEIR** et **Jeu de la vie**. Les résultats montrent que les leviers agissant sur la **localité mémoire** et la **granularité du parallélisme** produisent les gains les plus significatifs, tandis que les micro-optimisations apportent des améliorations plus modestes et souvent dépendantes du contexte.

7.2 Tendances et interprétation technique

- **Optimisations systématiquement bénéfiques**
  Le niveau de compilation **O3** et, pour la multiplication dense, la **transposition** se distinguent comme des choix sûrs. O3 offre un bon compromis stabilité/performance ; la transposition améliore fortement la localité pour les grands problèmes (gains observés jusqu'à $\times\mathbf{8}$ sur certains cas pour chacune des deux optimisations).

- **Effets contextuels et mécanismes**
  Blocking et **vectorisation** présentent des comportements variables selon l'application et l'implémentation. Le blocking peut donner des gains importants sur la multiplication matricielle mais être neutre ou nuisible sur des codes où l'accès est déjà local (Jeu de la vie) ou mal calibré. La vectorisation automatique du compilateur est généralement suffisante ; la vectorisation manuelle n'a pas apporté d'amélioration notable dans nos tests.
  Techniquement, ces observations s'expliquent par la réduction des stalls mémoire et des défauts de cache (transposition, blocking) et par la capacité à remplir efficacement les registres SIMD (vectorisation) lorsque la granularité et l'absence de dépendances le permettent.

- **Parallélisme partagé vs distribué**
  OpenMP est très efficace lorsque la parallélisation s'intègre naturellement (boucles indépendantes, travail équilibrable) ; il offre un excellent rapport effort/gain sur mono-nœud. Lorsqu'il impose des restructurations lourdes (copies, verrous, synchronisations), il peut dégrader les performances. **MPI** apporte des gains temporels réguliers et tend à être économe en énergie CPU, mais sa mise en œuvre est la plus exigeante et la surcharge de communication limite la scalabilité.

7.3 Recommandations pratiques

1. **Priorité 1** : appliquer **O3** et tester la **transposition** pour codes à accès matriciels.

2. **Priorité 2** : tester **OpenMP** quand la parallélisation est simple ; combiner avec blocking si la localité s'améliore.

3. **Priorité 3** : évaluer le **blocking** systématiquement (petit/moyen/grand) et automatiser la recherche de la taille de bloc optimal.

4. **Priorité 4** : activer la vectorisation automatique et comparer temps/énergie ; de gros gains sur au moins l'une des deux données peux être atteinte.

5. **MPI** : réserver MPI aux scénarios nécessitant distribution ; soigner la topologie et minimiser les échanges.

7.4 Note personnelle

D'après l'expérience acquise pendant le stage, quelques règles empiriques se dégagent et aident à prioriser les efforts :

- **Privilégier la simplicité** : les optimisations simples et évidentes (O3, transposition pour accès matriciels, ajout d'OpenMP quand c'est trivial) se sont souvent révélées les plus efficaces. Les restructurations lourdes (réécriture pour blocking, insertion d'OpenMP dans des zones à dépendances) demandent beaucoup de temps et d'effort pour des gains parfois marginaux.

- **OpenMP comme indicateur** : si l'intégration d'OpenMP est **facile**, c'est souvent le signe d'un **gros gain potentiel** ; si elle impose des contournements complexes, mieux vaut explorer d'autres pistes.

- **MPI et énergie** : MPI s'est avéré régulièrement **économe en énergie CPU** et apporte un gain temporel, mais reste coûteux en développement et sujet à des bugs subtils.

- **Vectorisation au cas par cas** : la vectorisation automatique suffit la plupart du temps ; la vectorisation forcée peut parfois réduire l'énergie même sans accélérer l'exécution. Tester systématiquement reste la meilleure pratique.

7.5 Limites et perspectives

- **Limites** : les mesures proviennent d'une **seule machine mono-nœud** et d'un compteur RAPL ciblant principalement le package CPU ; la consommation DRAM peut être sous-estimée. Certains gains chiffrés (×8, 11,6 s vs 35 s, etc.) sont représentatifs des configurations testées et doivent être interprétés avec la configuration associée (taille, block, flags).

- **Perspectives** : étendre les tests à d'autres architectures et à des scénarios multi-nœuds ; automatiser la calibration des paramètres (taille de bloc, affinité des threads) ; mesurer la consommation DRAM et compléter RAPL par d'autres compteurs matériels ; documenter plus finement les variantes d'implémentation (mlt*, SUMMA/Fox/Cannon).

7.6 Bilan final

Ce travail illustre la démarche expérimentale en HPC : formuler des hypothèses, mesurer systématiquement, interpréter et prioriser. Les conclusions sont pragmatiques : **commencer par les optimisations à faible coût d'implémentation**, mesurer les effets sur temps et énergie, puis approfondir seulement les pistes qui montrent un vrai potentiel. Avec les harmonisations mineures proposées (répétitions, légendes, renvois aux configurations pour les chiffres clés), le rapport est prêt à fournir des recommandations opérationnelles et pédagogiques utiles pour des développements futurs.

Annexes

Table 16: Résumé des ratios temps/énergie pour mpi_C2_1D_NV_B4 et mpi_C2_1D_VF_B4 vs mpi_B0

exec	mean_speedup	mean_energy_ratio	n
mpi_B0	1.0000000	1.000000	12
mpi_C2_1D_NV_B4	0.3714758	6.156458	12
mpi_C2_1D_VF_B4	1.1220863	1.486602	12

Table 17: Consommation totale et moyenne d'énergie par algorithme (np=16)

exec	total_energy	mean_energy	n
mpi_B0	52.95770	4.413141	12
mpi_C2_1D_NV_B4	281.78818	23.482348	12
mpi_C2_1D_VF_B4	54.42731	4.535609	12

Table 18: Énergie par taille et par algorithme (np=16)

exec	size	total_energy	mean_energy	n
mpi_B0	256	1.193923	0.3979744	3
mpi_B0	512	2.455006	0.8183354	3
mpi_B0	1024	10.265906	3.4219686	3
mpi_B0	2048	39.042861	13.0142869	3
mpi_C2_1D_NV_B4	256	5.219555	1.7398515	3
mpi_C2_1D_NV_B4	512	15.729304	5.2431015	3
mpi_C2_1D_NV_B4	1024	56.320394	18.7734645	3
mpi_C2_1D_NV_B4	2048	204.518924	68.1729748	3
mpi_C2_1D_VF_B4	256	2.162907	0.7209691	3
mpi_C2_1D_VF_B4	512	4.649918	1.5499727	3
mpi_C2_1D_VF_B4	1024	10.251691	3.4172305	3
mpi_C2_1D_VF_B4	2048	37.362790	12.4542633	3

Table 19: Résumé des ratios temps/énergie pour omp_C2_1D_NV_B4 et omp_C2_1D_VF_B4 vs omp_C2_1D_B4 (threads = 4, 8, 16)

exec	mean_speedup	mean_energy_ratio	n	threads
omp_C2_1D_B4	1.0000000	1.0000000	12	4
omp_C2_1D_NV_B4	1.0691055	0.9360755	12	4
omp_C2_1D_VF_B4	1.0136054	0.9842063	12	4
omp_C2_1D_B4	1.0000000	1.0000000	12	8
omp_C2_1D_NV_B4	1.0447460	0.9575767	12	8
omp_C2_1D_VF_B4	1.0119363	0.9872769	12	8
omp_C2_1D_B4	1.0000000	1.0000000	12	16
omp_C2_1D_NV_B4	0.9581209	1.1348288	12	16

exec	mean_speedup	mean_energy_ratio	n	threads
omp_C2_1D_VF_B4	0.9368407	1.1185044	12	16

Table 20: Consommation totale et moyenne d'énergie par algorithme (threads=8)

exec	total_energy	mean_energy	n
omp_C2_1D_B4	261.8173	21.81811	12
omp_C2_1D_NV_B4	258.2161	21.51800	12
omp_C2_1D_VF_B4	262.8945	21.90788	12

Annexe

Annexe SEIR_A

